CLEAN FEATURE ANCHORING: MITIGATING THE ACCURACY–ROBUSTNESS TRADE-OFF BY SELF-DISTILLING A NATURALLY TRAINED MODEL

Anonymous authors
Paper under double-blind review

ABSTRACT

Adversarial training significantly improves adversarial robustness, but robust models attain substantially lower standard accuracy than naturally trained ones. This accuracy gap has motivated a line of work that brings a naturally trained network into adversarial training. Existing methods distill its logits into the robust model or add its predictions as an auxiliary regularizer beside the label loss. We also employ a naturally trained network, but we exploit it far more directly: the label loss is discarded entirely and the backbone is trained against the natural network alone. In this paper, we propose Clean Feature Anchoring (CFA), which anchors the adversarial feature of the student to the clean feature of the teacher rather than to its logits. We prove that this single objective bounds both fidelity to the teacher and local stability of the student up to a constant factor, so that no balancing coefficient is required. We further assign the attack radius per sample according to the input sensitivity of the loss under a fixed total budget, which concentrates the anchor where the feature is most fragile. Experimental results show that CFA recovers standard accuracy without sacrificing robustness. Particularly, CFA attains 62.17% standard accuracy at 28.86% AutoAttack accuracy on CIFAR-100 with the ResNet-18 architecture, and consistent improvements are observed on CIFAR-10 and Tiny-ImageNet.

1 INTRODUCTION

Deep neural networks are vulnerable to adversarial examples, which are imperceptible perturbations of the input that change the prediction (Goodfellow et al., 2014; Madry et al., 2017; Carlini & Wagner, 2017). Such vulnerability raises concerns wherever these models are deployed in safety-critical systems (Ma et al., 2021; Grigorescu et al., 2020). Among the defenses proposed in response, adversarial training (Madry et al., 2017) remains the most reliable under strong evaluation (Athalye et al., 2018; Croce & Hein, 2020), as the model is optimized directly on perturbations produced by an inner maximization. The robustness is obtained at a cost in standard accuracy. An adversarially trained network classifies unperturbed inputs considerably worse than the same architecture trained naturally, and the gap widens as the perturbation budget grows. This accuracy–robustness trade-off is partly intrinsic to the objective (Tsipras et al., 2018; Zhang et al., 2019) and remains a major limitation of adversarially trained models.

Two lines of work reduce the trade-off by changing what the robust model is trained to predict. The first replaces the one-hot label with a softer target inside adversarial training. A hard label asserts full confidence for every point of an ϵ-ball and is therefore a poor supervisory signal for perturbed inputs. Label smoothing (Pang et al., 2020), consistency regularization (Tack et al., 2022) and annealing self-distillation rectification (Wu et al., 2024) construct such a target without any additional network, and they alleviate both the trade-off and robust overfitting (Rice et al., 2020). The second line, adversarial distillation, instead supplies the target from an adversarially trained teacher. ARD (Goldblum et al., 2020), IAD (Zhu et al., 2021), RSLAD (Zi et al., 2021), AdaAD (Huang et al., 2023) and IGDM (Lee et al., 2025) differ in which teacher quantity is matched and at which input the teacher is evaluated, and together they report the strongest accuracy–robustness

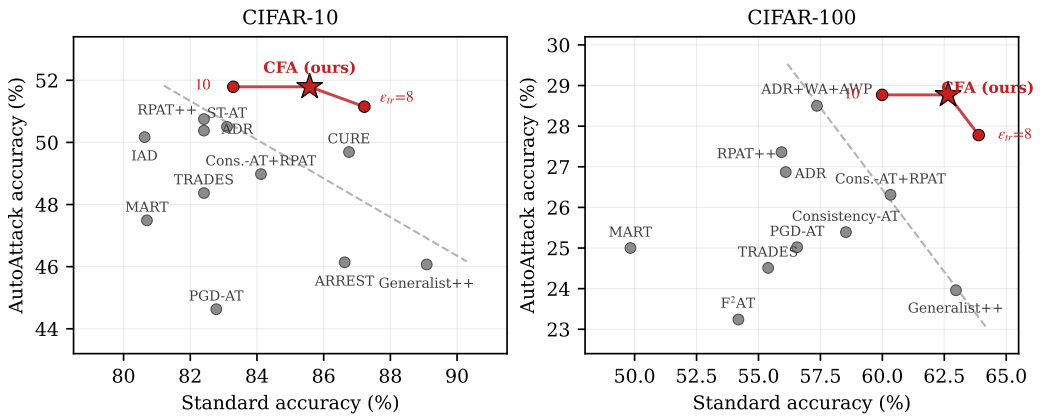

Figure 1: The accuracy–robustness frontier on ResNet-18 against ℓ_∞ perturbations of $\epsilon = 8/255$. Grey points are published results (Table 17) and the dashed line is a least-squares fit through the Pareto-optimal ones. CFA is drawn as a curve because the training radius moves the method along the frontier; the star marks the configuration used in all other experiments.

frontiers for small architectures. Their guidance nevertheless presupposes a network that is already robust. In current practice that network is a WideResNet adversarially trained on large volumes of generated data (Wang et al., 2023b), which is more expensive to obtain than the student it teaches.

A naturally trained copy of the student architecture holds precisely the standard accuracy that adversarial training gives up, and it is available at no adversarial cost. Such a network is therefore an appealing source of guidance for the accuracy axis. Several methods already use such a network. ARREST (Suzuki et al., 2023) initializes the robust model from a naturally pretrained one and adds a representation-distance term to the label loss. DP-FAT (Cho & Kim, 2026) distills the natural teacher after rescaling its logits per sample, so that only the class-discriminative direction is transferred. B-MTARD (Zhao et al., 2024) pairs a natural teacher with a robust one so that each supervises the examples it handles well. In all of these the natural network contributes an auxiliary term whose weight against the label loss has to be tuned, and the guidance is read from the teacher's logits. We also employ a naturally trained network, but we exploit it far more directly, by removing the label loss and training the backbone against the natural network alone.

Whether such a substitution is possible depends on which quantity of the teacher is read. Adversarial distillation attributes the difficulty of the one-hot target to its sharpness and responds by softening it (Chen & Lee, 2021; Zi et al., 2021; Wu et al., 2024; Zhao et al., 2024). A naturally trained teacher is the extreme point of that prescription, since its predictions are far more confident than those of an adversarially trained model. Softening the target nevertheless improves the wrong quantity. Over a temperature sweep the student gains AutoAttack accuracy while its standard accuracy stays where adversarial training leaves it, even though the teacher supplying the target is close to twenty points more accurate (Section 3). The same conclusion follows from running the published distillation objectives on our natural teacher, where ARD, RSLAD and AdaAD all fall below plain adversarial training initialized from that teacher. The accuracy a natural teacher holds is therefore not recoverable through its softmax, and we read the teacher one layer earlier instead.

In this paper, we propose **Clean Feature Anchoring** (CFA), which trains the backbone by matching the student's feature at the adversarial input to the teacher's feature at the clean input. The label term is removed rather than supplemented, so the anchor is the entire objective for the backbone, and the classifier is inherited from the teacher and left untrained. The teacher is evaluated only outside the perturbation ball. Its behaviour under attack therefore never enters the optimization, and all robustness is left to the inner maximization of the student, as in standard adversarial training. We do not claim that a natural teacher is preferable to a robust one for the robustness axis. Our claim is that guidance for the accuracy axis can be obtained without paying for a robust teacher.

Removing the label term would be a poor trade if the remaining term only held the student near its teacher. We show instead that the single anchor term controls two properties at once. Writing F for

the clean deviation of the student from its teacher and O for the diameter of the student's output over the ϵ-ball, the anchor loss L satisfies $L \leq F + O \leq 3L$. Minimizing L therefore bounds teacher fidelity and local stability together up to a constant factor, and no coefficient is needed to balance them. The argument uses only the triangle inequality and consequently applies to the networks we train rather than to a model of them.

We further assign the attack radius per sample rather than uniformly, in order to equalize what the attack accomplishes on the quantity the objective actually measures. The radius is optimized over pixels while the anchor is a displacement in feature space, so a fixed pixel radius moves different examples by very different amounts in the space that matters. **Sensitivity-matched** ϵ redistributes the radius in inverse proportion to the input sensitivity of the loss and holds the total budget over the batch fixed. A comparison against a uniform radius then isolates the allocation from the strength of the attack.

Our analysis of the teacher yields a second observation. Across a ladder of naturally trained teachers that differ only in training length, teacher accuracy and student accuracy are almost perfectly anti-correlated ($r = -0.999$). The most accurate teacher produces the weakest student. What the student inherits is the class geometry of the teacher rather than its predictions, and the two quantities are not monotone in one another. The training length of the teacher is therefore a control on the accuracy–robustness frontier that costs nothing to exercise. The corresponding control for a robust teacher is the expensive part of obtaining one. Non-monotonicity of this kind is also reported in the robust-teacher setting, where a more robust teacher may fail to improve the student (Anonymous, 2026).

The resulting objective carries no temperature, no loss weight and no auxiliary head, and an identical configuration is applied to every dataset. On CIFAR-100 with ResNet-18, CFA attains 62.17% standard accuracy at 28.86% AutoAttack accuracy (Figure 1). The highest published AutoAttack accuracy in this setting is 28.50% at 57.36% standard accuracy and the highest published standard accuracy is 62.97% at 23.96% AutoAttack accuracy (Table 17), so CFA improves on the former by 4.81 points of standard accuracy and on the latter by 4.90 points of AutoAttack accuracy. Consistent improvements are observed on CIFAR-10 and Tiny-ImageNet under the same configuration. Our contributions are summarized as follows:

- We show that a naturally trained network can supervise adversarial training on its own when it is read at the clean input and at the feature layer, and we prove that the resulting single term bounds fidelity to the teacher and local stability of the student up to a constant factor, so that no coefficient balances the two.
- We show that a natural teacher transfers class geometry rather than accuracy, with teacher and student accuracy anti-correlated at $r = -0.999$ across teachers, which turns the training length of the teacher into a trade-off control that is available only in this setting.
- We propose Clean Feature Anchoring together with sensitivity-matched ϵ, which requires no loss weight, no temperature and no per-dataset tuning. Robustness remains attributable to the inner maximization of the student, and the choice of what that maximization ascends is consequential: without weight averaging, AWP or the radius rule, the anchor exceeds label cross-entropy by 4.45 points of AutoAttack accuracy, measured against cross-entropy at its own optimizer and schedule rather than ours.
- We establish a new accuracy–robustness frontier on CIFAR-10, CIFAR-100 and Tiny-ImageNet, and we show that the gain does not come from weight averaging or AWP, since CFA with weight averaging alone and half the schedule already exceeds the best published result that uses both.

2 Related Work

To be written.

3 Analysis

Adversarial training trades clean accuracy for robustness, and this section asks where in its construction the trade sits. The answer we arrive at is that it is in the target: a label asserted over a whole ball

is a demand that cannot be met without distorting the clean prediction, and replacing it with a fixed point in feature space turns two competing objectives into one. That claim is what Proposition 1 makes precise, and it is also what tells us the anchor cannot cost robustness — which is a weaker statement than the paper needs, and we say so where it matters.

3.1 PRELIMINARIES

Let $f = h \circ \Phi$ be a classifier with feature map Φ and linear head h. Adversarial training (Madry et al., 2017) solves

$$\min_{\theta} \mathbb{E}_{(x,y)} \max_{x' \in B(x,\epsilon)} \ell\big(f_\theta(x'), y\big), \qquad B(x,\epsilon) = \{x' : \|x' - x\|_\infty \le \epsilon\}. \tag{1}$$

The supervisory signal is the label, and it is asserted uniformly over $B(x, \epsilon)$: every point of the ball must reach full confidence in one class. That is where the accuracy cost comes from. We write $f_t = h_t \circ \Phi_t$ for a network of the same architecture trained naturally on the same data—the teacher throughout, frozen—so that $\Phi_t(x)$ is its feature and $f_t(x)$ its logits, and ask what it can supply.

3.2 WHERE A NATURAL TEACHER FAILS, AND WHERE IT DOES NOT

A method that consumes a teacher can go wrong in two independent places: *where* it reads the teacher, and *what* it reads from it. Both defects are present when a naturally trained network is used the way adversarial distillation uses one, and they can be measured separately.

What is read: the field's prescription is softening, and it is only half a fix. Four separate lines of work converge on the same diagnosis—the target is too sharp—and each builds its own machinery to soften it. LTD (Chen & Lee, 2021) argues that one-hot labels are the wrong supervision for adversarial training and replaces them with soft ones. RSLAD (Zi et al., 2021) makes the smoothness of the teacher's logits its central claim. ADR (Wu et al., 2024) draws its target from a weight average and scales it by a softmax temperature that starts near-uniform and is annealed down over training. B-MTARD (Zhao et al., 2024) formalizes the quantity being controlled—writing the teacher's knowledge scale as $\log C - H(P_T)$, so that a sharper teacher carries a larger one—and adjusts each teacher's temperature online to equalize it. In each of these the operative object is $\mathrm{softmax}(z_t/\tau)$ and the design question is τ.

Not every method that consumes a natural network poses it, and Section 2 places the exceptions; LBGAT (Cui et al., 2021) in particular matches logits under a squared error with no softmax at all, at the cost of training its natural branch jointly.

By that account a naturally trained teacher is the extreme case, and the measurement agrees: on clean test data its maximum softmax probability is 0.820, against 0.016 for the adversarially trained student it is supposed to teach. It is asking for a confidence twenty times sharper than the student's own.

The prescription follows, and it works. Sweeping τ on that teacher's logits in a fixed base regime—50 epochs, no weight averaging, no AWP, $\epsilon = 8/255$—AutoAttack accuracy rises from 20.84 at $\tau = 1$ to 24.48 at $\tau = 4$, then turns over. Sharpness is a genuine defect and softening genuinely repairs it.

target	Clean	AA	NRR
teacher logits, $\tau = 1$	58.26	20.84	30.70
teacher logits, $\tau = 4$	59.39	24.48	34.67
teacher logits, $\tau = 16$	57.78	24.00	33.91
teacher feature	**62.72**	**25.88**	**36.64**

What it does not repair is the accuracy. Across the whole sweep clean accuracy moves between 57.78 and 59.39—it is flat in τ—while the teacher it is being distilled from has 77.66. Reading the same teacher's *feature* instead, with no temperature anywhere, gains a further 1.40 AutoAttack and 3.33 clean, at the same read point and under the same regime.

The two numbers separate cleanly. **Temperature moves robustness and leaves accuracy where it was; the feature moves both.** Sharpness is therefore a real problem with a real fix, and it is not

the problem this paper is trying to solve: the accuracy a naturally trained network holds does not survive its own softmax, and no choice of τ puts it back. It survives one layer earlier.

Where it is read: off the clean point the teacher has nothing to say. Under $\epsilon = 8/255$ the teacher's own feature rotates by 63.8° and its norm inflates by $\times 2.45$, so its output inside the ball is not a softened version of its output at the centre. The consequence is sharpest for methods that read it there. IGDM (Lee et al., 2025) matches a finite difference $f_t(x{+}\delta) - f_t(x{-}\delta)$ as a stand-in for the teacher's input gradient, which is a gradient only while the teacher is locally linear; its own diagnostic reports a Taylor remainder proportion of 0.012 for adversarially trained teachers. Applying that diagnostic to our checkpoints gives 0.0111 for an adversarially trained ResNet-18—reproducing their value, so the measurement is calibrated—and $\mathbf{0.4763}$ for the natural teacher, forty times further from linear. The target it constructs is correspondingly degenerate: $\|f_t(x{+}\delta) - f_t(x{-}\delta)\| = 14.14$ against $\|f_t(x)\| = 18.36$, with the softmax of that difference placing 0.917 of its mass on one class.

Together, they cost more than the teacher is worth. Table 5 gives the published objectives with our natural teacher in place of the robust one they assume, each at its own published recipe. Every one lands below plain adversarial training initialized from that same teacher—that is, below not distilling at all. The accuracy the teacher holds is real, and the logit route does not recover it.

What follows keeps the read point at x, where the second defect cannot arise, and takes the features, which the first measurement shows to be the better read.

3.3 What a Clean-Feature Anchor Controls

Consider matching the teacher's feature at the *clean* input against the student's feature at the *adversarial* input, that is, minimizing

$$\mathcal{L}(x) \;=\; \max_{x' \in B(x,\epsilon)} \big\|\Phi_s(x') - \Phi_t(x)\big\|^2. \tag{2}$$

Read literally, Equation (2) asks the student to reproduce a *fixed* vector from every point of the ball. That is a fidelity constraint, and a fidelity constraint alone would have no reason to produce robustness. It turns out not to be one, and the reason is visible once the quantity is compared with the two properties one would otherwise have to impose separately.

Fix x, write $B = B(x, \epsilon)$, and define

$$L = \max_{x' \in B} \|\Phi_s(x') - \Phi_t(x)\|, \tag{3}$$

$$F = \|\Phi_s(x) - \Phi_t(x)\|, \tag{4}$$

$$O = \max_{x', x'' \in B} \|\Phi_s(x') - \Phi_s(x'')\|. \tag{5}$$

Here L is the square root of Equation (2), the quantity the method minimizes; F is *fidelity*, how far the student is from the teacher at the clean point; and O is *oscillation*, the diameter of the student's own feature map over the ball. F is what adversarial training gives up relative to natural training, and O is what its label term buys. They are ordinarily two objectives.

Proposition 1. *For every x and every $\epsilon \geq 0$,*

$$L \;\leq\; F + O \;\leq\; 3L. \tag{6}$$

Proof. For the right-hand inequality, $F \leq L$ because $x \in B$, so the maximum in Equation (3) is at least the value at x. For O, insert $\Phi_t(x)$ between the two student features: for any $x', x'' \in B$,

$$\|\Phi_s(x') - \Phi_s(x'')\| \;\leq\; \|\Phi_s(x') - \Phi_t(x)\| + \|\Phi_t(x) - \Phi_s(x'')\| \;\leq\; 2L,$$

and taking the maximum over x', x'' gives $O \leq 2L$. Adding the two bounds gives $F + O \leq 3L$. For the left-hand inequality, insert $\Phi_s(x)$ instead: for any $x' \in B$,

$$\|\Phi_s(x') - \Phi_t(x)\| \;\leq\; \|\Phi_s(x') - \Phi_s(x)\| + \|\Phi_s(x) - \Phi_t(x)\| \;\leq\; O + F,$$

and taking the maximum over x' gives $L \leq F + O$. □

Only the triangle inequality is used, so Proposition 1 holds for the networks we actually train: no smoothness, no linearity, no assumption on the architecture or on the attack that realizes the maximum. Two consequences follow immediately.

Corollary 1. *If the anchor is driven to $L \leq \delta$ at x, then $F \leq \delta$ and $O \leq 2\delta$ at x.*

That is the operative statement: driving one scalar down bounds *both* properties, at rates that differ by a factor of two and by nothing else. Conversely, by the left inequality, L cannot be small unless both are, so the objective admits no solution that satisfies one at the expense of the other. A weighted sum $F + \lambda O$ has no such property for any fixed λ: its level sets contain points where either term is arbitrarily large. This is why the anchor is stated as the whole objective rather than as a regularizer, and why there is no coefficient to select.

Where the constant 3 comes from, and how tight it is. The factor is $1 + 2$: one from $F \leq L$ and two from $O \leq 2L$, and the second is attained when the student's outputs at the two extreme points of the ball sit on opposite sides of $\Phi_t(x)$ at distance L each. Our trained students are far from that configuration, which is the content of the measurement below.

The teacher appears at x and nowhere else. Equation (2) evaluates Φ_t at the clean point only, in the loss and in the attack that realizes the maximum. Whatever the teacher does inside B is therefore absent from the objective, and the instability measured in Section 3.2—63.8° of rotation, $\times 2.45$ of norm—has no path into the optimization. The same fact settles the symmetric question: since the teacher is never asked to resist a perturbation, it cannot supply robustness either, and all of it is produced by the student's inner maximization.

This is testable by moving the read point and changing nothing else, and the test is decisive. Replacing $\Phi_t(x)$ with $\Phi_t(x')$ in both the loss and the attack drives AutoAttack accuracy from 26.19 to 0.00 while clean accuracy *rises* to 76.11, within 1.55 of the natural teacher (Table 7). The failure is not that a moving target is harder to track. It is that $\|\Phi_s(x') - \Phi_t(x')\|$ is minimized exactly by $\Phi_s = \Phi_t$, so the student that copies a natural teacher everywhere is optimal—and it has no robustness at all. The clean read point is what makes Equation (2) a non-degenerate objective, and the same asymmetry that keeps the teacher's fragility out is what keeps the trivial solution out.

Verification. Proposition 1 predicts that a student trained on L ends up with a small O, and the prediction is checkable on the trained network. Measured over $\epsilon = 8/255$ balls on CIFAR-100, $L = 7.58$, $F = 6.30$ and $O_{\text{lb}} = 2.16$, against the teacher's own $O = 6.68$: the student is $3.1\times$ more stable than the network it was distilled from, on the same balls. Two things follow. The teacher's instability was not inherited, which is what the construction predicts. Furthermore, $F \gg O_{\text{lb}}$, so what remains in the loss at convergence is fidelity rather than instability—the anchor is not being satisfied by making the student flat.

What the proposition is for, and what it is not. Proposition 1 says the anchor does not *cost* robustness: local stability is bounded by the same scalar as fidelity, so the two cannot be traded against each other. It is not a lower bound on how much robustness the objective buys, and we do not read it as one; Appendix B gives two partial results in that direction, each with the reason it falls short.

The measurement is stronger than the proposition, and it is stronger for a different reason than the proposition would suggest. Asked to work with neither weight averaging nor AWP nor a sensitivity-matched radius—the anchor alone, against label cross-entropy under an identical schedule, initialization and training radius—the anchor leads by 6.69 clean *and* 5.91 AutoAttack, and by 4.63 and 4.45 against cross-entropy run at its own optimizer and schedule instead of ours (Table 7, Table 14). The obvious reading is that Proposition 1's bound on O is doing the work, and Section 5.4 shows it is not: the two students oscillate by the same relative amount under a matched attack. What differs is class separation, by a factor of 5.5, and that is where we locate the effect. For scale, the full recipe leads cross-entropy *with* weight averaging and AWP by 2.40. So the anchor is a source of robustness rather than a neutral passenger on one, and at matched stack it is the larger source; the components in Table 6 are additive on top of it rather than the origin of the effect. We keep Proposition 1 stated as the weaker claim because that is what the inequality supports—the gap is measured, not derived.

4 METHOD

4.1 CLEAN FEATURE ANCHORING

Adversarial training requires the model to be confidently correct at every point of a ball around every training example, and that requirement is the source of its cost in standard accuracy. Methods that address the trade-off keep the requirement and soften the target it is imposed on, whereas we replace the target itself. The backbone is trained against a single fixed vector, namely the feature that a naturally trained network of the same architecture assigns to the clean image. The perturbation ball is then asked to map onto that vector rather than onto a confident label. The network supplying the vector is not robust, which is a deliberate choice: it provides a target that is accurate, and the robustness is produced by the maximization that finds x_{adv}. Such a division of labour is only coherent once the label loss is removed, because a label loss and an inaccurate-but-robust signal would otherwise compete for the same parameters. The remainder of this subsection states the objective and three properties that follow from its form rather than from measurement, and Section 4.2 treats the one quantity that is left to choose, namely how large a ball each example receives.

Let Φ_t be the frozen feature map of a network of the same architecture trained naturally on the same data, and let h_t be its classifier. The student's feature map is initialized at $\Phi_s \leftarrow \Phi_t$ and trained with

$$\mathcal{L} \;=\; \mathbb{E}_x \big\|\Phi_s(x_{\mathrm{adv}}) - \Phi_t(x)\big\|^2, \qquad x_{\mathrm{adv}} \;=\; \underset{x' \in B(x,\epsilon)}{\arg\max} \big\|\Phi_s(x') - \Phi_t(x)\big\|, \tag{7}$$

where the inner maximization is realized by K steps of projected gradient ascent on the same quantity the outer problem minimizes,

$$x^{(k+1)} \;=\; \Pi_{B(x,\epsilon)}\Big(x^{(k)} + a \cdot \operatorname{sign} \nabla_x \big\|\Phi_s(x^{(k)}) - \Phi_t(x)\big\|^2\Big), \qquad x^{(0)} = x + \eta, \tag{8}$$

with η a small random initialization and Π the projection onto the ball. The target $\Phi_t(x)$ is a constant of this ascent, since it is computed once at the clean input and does not move as $x^{(k)}$ does. The inner and outer problems therefore optimize the same quantity, as they do in standard adversarial training and unlike methods whose attack is generated against a different objective from the one being trained. Equation (7) is the entire objective for the backbone, without a label term, a logit term or a coefficient joining them, and the three properties below are consequences of that form.

(i) The teacher never enters the perturbation ball. The teacher is evaluated at x alone in both Equation (7) and Equation (8), so the objective depends on Φ_t only through the single vector $\Phi_t(x)$. Two teachers that agree at x and differ arbitrarily on $B(x,\epsilon) \setminus \{x\}$ consequently induce the same optimization problem, and the rotation that the teacher undergoes under attack (Section 3) is exactly such a difference. The same argument runs in the other direction as well, because a quantity the objective cannot observe cannot serve as a source of robustness either. The restriction is load-bearing rather than convenient. Evaluating Φ_t at x' instead would make $\Phi_s = \Phi_t$ a global minimizer of Equation (7), and that minimizer is a natural model; the student trained this way collapses onto the teacher and reaches 0.00 AutoAttack accuracy (Table 7). Restricting the teacher to x therefore keeps the fragility of the teacher out of the objective and keeps the objective from admitting a degenerate solution.

(ii) The anchor is exactly zero at the clean point at initialization. The initialization $\Phi_s \leftarrow \Phi_t$ gives $\|\Phi_s(x) - \Phi_t(x)\| = 0$ for every x, so the loss is nonzero only because of the perturbation. Every gradient the backbone receives at the start of training is thus attributable to the attack, which is the quantity we intend to train against. A logit target has the same property only at temperature 1, where it carries no dark knowledge. The property is also used by Section 4.2: the per-sample sensitivities are identical at the first step, so the radius allocation begins uniform and differentiates only as the student moves away from its teacher.

(iii) The classifier is inherited, which is what a feature target buys. The student at test time is $h_t \circ \Phi_s$, and the head is never trained. Inheriting the head is possible because the target is written on the feature. A loss written on logits constrains only the composition $W\Phi$, and it therefore determines the pair (Φ, W) only up to the gauge

$$(\Phi, W) \;\longmapsto\; (A\Phi,\; WA^{-1}), \qquad A \in \mathrm{GL}(d), \tag{9}$$

so that the same logits are reachable from a continuum of representations, none of which the teacher's classifier is calibrated for. Fixing the target at the feature removes this freedom and pins Φ_s to Φ_t itself, which is what keeps h_t the correct classifier for it. Table 9 shows that inheriting the head is preferred rather than merely permissible, since every alternative we tried scores below leaving the classifier alone, including the best head temperature. Inheritance also removes the last two hyperparameters, as a head distillation term would reintroduce both a temperature and a weight.

4.2 SENSITIVITY-MATCHED ϵ

The problem a uniform radius creates. Equation (7) is optimized over a pixel ball while the quantity it measures is a feature displacement, and the two are not proportional. A single ϵ therefore moves the objective by an amount that varies substantially across the dataset, and the coefficient of variation of $\|\nabla_x \mathcal{L}\|$ within a minibatch is 0.64 when logged over training. Examples that receive the same pixel budget are consequently attacked with very different force in the geometry the loss is written in, which is not something Equation (7) asks for.

Equalizing the movement. We assign the radius per sample in order to equalize how far the attack moves the objective rather than how far it moves the input. The first-order change of a differentiable loss inside an ℓ_∞ ball of radius e is

$$\max_{\|\delta\|_\infty \le e} \langle \nabla_x \mathcal{L}, \delta \rangle \;=\; e\, \|\nabla_x \mathcal{L}\|_1, \tag{10}$$

since ℓ_1 is the dual norm of ℓ_∞. Writing $g_i = \|\nabla_x \mathcal{L}(x_i)\|_1$, the movement induced at example i by a radius ϵ_i is $\epsilon_i g_i$ to first order. Equalizing that movement across a minibatch of N examples amounts to imposing

$$\epsilon_i\, g_i \;=\; c \quad \text{for all } i, \qquad \text{subject to} \qquad \sum_{i=1}^{N} \epsilon_i \;=\; N\epsilon. \tag{11}$$

The constraint holds the total attack budget fixed, which is what makes the rule comparable to a uniform radius at equal attack strength. Solving Equation (11) gives $c = N\epsilon \big/ \sum_j g_j^{-1}$ and hence

$$\epsilon_i \;=\; N\epsilon \cdot \frac{g_i^{-1}}{\sum_j g_j^{-1}}, \qquad \text{generalized to} \qquad \epsilon_i \;\propto\; g_i^{-p}, \tag{12}$$

where $p = 0$ recovers the uniform radius and $p = 1$ gives the full equalization above. The allocation of Equation (12) is equivalently the max–min solution, because moving budget from the example whose objective moves most to the example whose objective moves least improves the worst case until no such move remains. Our implementation reads the gradient in ℓ_2 rather than in the ℓ_1 derived here, for the reason given in Appendix D, and the two norms differ by 0.29 points of AutoAttack accuracy. What the rule gains over a uniform radius is preserved in direction under either norm and is larger on the clean axis than on the robust one, and Appendix D reports both settings.

Keeping the radii in range. Equation (12) assigns unbounded radii where $g_i \to 0$, which is the situation at the start of training by property (ii). We therefore constrain $\epsilon_i/\epsilon \in [0.5, 1.5]$ and solve the budget constraint inside that box, which Appendix D states exactly and which reduces to restoring the mean when no clip is active. We use $p = 1$ throughout and never tune it, and Table 11 reports the $p = 0$ endpoint under the same recipe.

Relation to existing per-sample radii. Assigning a different ϵ to each example is not new, as IAAT (Balaji et al., 2019), MMA (Ding et al., 2020) and CAT (Cheng et al., 2022) all do so. These methods assign the radius from the difficulty of the example or from its margin in input space, whereas Equation (12) assigns it from the input sensitivity of the training loss, which is the geometry the objective is written in. The difference is not one of dispersion, since all three signals are dispersed on the shipped recipe and ours is in fact the least dispersed of them (Appendix D). What separates the rules is the geometry along which the budget is spread, and the experiments in Section 5 therefore compare competing allocations rather than an allocation against its absence.

The distinction can be tested without confounding it with the budget. We hold the exact multiset of weights that Equation (12) produces, so that the values, the clip and the mean are unchanged, and

only reassign which example receives which weight in the order that IAAT and CAT would use. The reassignment alone costs 0.80 points of AutoAttack accuracy and falls below the uniform baseline on AutoAttack, CW and NRR alike. Replacing the signal outright by difficulty magnitude or by the logit margin of MMA is likewise worse than allocating nothing (Table 11). Among the signals we tested, only the input sensitivity of the loss improves on a uniform radius.

Algorithm 1 states the procedure in full. Training the natural teacher is ordinary supervised training and is the only place where labels appear. The anchored phase uses no labels at all, because the teacher vector ϕ_i read once per example at the clean input is the entire supervisory signal, and the classifier is carried through untouched. The method introduces no loss weight, no temperature and no auxiliary head, and the teacher checkpoint is the only quantity we vary between datasets.

Caveats. Equation (10) is derived for the ℓ_1 norm while our runs read the gradient in ℓ_2, and the gap between the two is small but larger than our seed spread, as reported in Appendix D. The sensitivity g_i is also a single backward pass at the clean x_i, so it measures the sensitivity at the starting point of the attack rather than along its trajectory.

5 Experiments

This section establishes four claims. The first is the frontier claim itself, namely that the method gains standard accuracy without giving up robustness, measured against baselines we run rather than quote. The second is that the accuracy a natural teacher holds is not available to any method that distils from one, since the anchor would not be the contribution if the published objectives recovered that accuracy when handed our teacher. The third concerns the deletion of the label term, which has to be what produces the gain rather than the schedule, the initialization or the components stacked around it. The fourth is the mechanism, where we report a negative result: the explanation we expected is measurably false, and the explanation that survives concerns where features sit rather than how far they move.

5.1 Experimental Settings

Setup. CIFAR-10, CIFAR-100 (Krizhevsky et al., 2009) and Tiny-ImageNet-200 (Le & Yang, 2015), with random crop and horizontal flip only. For each dataset the teacher is a network of the same architecture trained naturally for 200 epochs on the same data (Table 1); no adversarial training enters its construction. The student is initialized from it and trained for 100 epochs with AdamW at learning rate 0.021 under a one-cycle schedule, batch size 128, with a 10-step PGD attack at step size $2/255$ and training radius $8.8/255$ (Section 5.5 varies it). Weight averaging ($\kappa = 0.999$, from 20% of the run) and an AWP proxy (Wu et al., 2020) ($\gamma = 0.005$, after epoch 10) are used and ablated in Section 5.4. Every dataset uses this identical configuration; only the teacher checkpoint changes. ResNet-18 throughout except Table 4.

Baselines and evaluation. Every baseline in Table 2 is run here rather than quoted: PGD-AT, TRADES and MART as standard adversarial training; ARD, RSLAD, AdaAD and IGDM as adversarial distillation, given our natural teacher in place of the robust one they assume; and RPAT++ reproduced from its own repository. Each row states in its *Teacher* column what it assumes available, cells we could not run are left empty with the reason, and published figures for all of them are collected in Table 17. We report clean accuracy and AutoAttack (standard version) on the full test set at $\epsilon = 8/255$, with Avg their mean and NRR their harmonic mean; per-attack values are in Table 16. All of our numbers are the final-epoch weight-averaged model, with no best-checkpoint selection.

5.2 Main Results

Baselines are grouped by what they assume is available: **(a)** standard adversarial training; **(b)** adversarial distillation from an *adversarially trained* teacher; **(c)** methods guided by a *naturally* trained network, the family ours belongs to; and **(d)** current accuracy–robustness trade-off methods. Published numbers carry their source; '–' means the paper does not report that cell for this dataset and architecture.

Table 1: Teacher models. AutoAttack accuracy is 0 by construction: these networks never see an adversarial example.

Dataset	Teacher	Epochs	Clean	AA
CIFAR-10	ResNet-18, natural	200	95.32	0.00
CIFAR-100	ResNet-18, natural	200	77.66	0.00
Tiny-ImageNet	ResNet-18, natural	200	66.29	0.00

Table 2: Main results. ResNet-18, ℓ_∞, $\epsilon = 8/255$. **Every number is measured in this framework** — one data pipeline, one attack implementation, one evaluation, the final-epoch weight-averaged model — so rows are comparable to each other rather than across papers. **Teacher** is what the method assumes available: – none, *self* its own weight average, *nat.* a naturally trained network. Avg is the mean of Clean and AA; NRR is their harmonic mean. Which variant each baseline runs, and why, is in Appendix H.

		CIFAR-10				CIFAR-100				
Method	T.	Clean	AA	Avg	NRR	Clean	AA	Avg	NRR	
PGD-AT	–	–	–	–	–	–	–	–	–	*other machine*
TRADES	–	–	–	–	–	–	–	–	–	*other machine*
MART	–	–	–	–	–	–	–	–	–	*other machine*
Consistency-AT	–	–	–	–	–	–	–	–	–	*other machine*
PGD-AT + WA	–	–	–	–	–	–	–	–	–	*other machine*
PGD-AT + WA + AWP	–	–	–	–	–	–	–	–	–	*other machine*
PGD-AT, init. at the teacher	nat.	82.23	50.72	66.47	62.74	57.73	26.46	42.09	36.29	
ARD	nat.	84.58	43.75	64.16	57.67	57.61	20.24	38.92	29.96	
RSLAD	nat.	85.15	42.91	64.03	57.06	59.68	21.30	40.49	31.40	
AdaAD	nat.	85.39	46.22	65.81	59.98	59.79	23.19	41.49	33.42	
AdaAD + IGDM	nat.	85.01	45.50	65.25	59.27	48.25	19.48	33.87	27.75	
Logit anchor + our stack	nat.	–	–	–	–	56.97	26.96	41.97	36.60	*C10 not run*
LBGAT	nat.	–	–	–	–	56.46	26.02	41.24	35.62	*C10 no recipe*
ARREST	nat.	–	–	–	–	–	–	–	–	*not run*
HAT	nat.	84.59	48.81	66.70	61.90	60.67	24.45	42.56	34.85	
ADR	self	–	–	–	–	–	–	–	–	*other machine*
CURE	self	–	–	–	–	–	–	–	–	*no repro.*
RPAT++	–	82.41	50.75	66.58	62.82	55.93	27.36	41.64	36.74	
CFA (ours)†	nat.	**84.96**	**51.74**	**68.35**	**64.31**	**62.17**	**28.86**	**45.52**	**39.42**	

Table 3: Tiny-ImageNet-200, ResNet-18. Same rows and protocol as Table 2; the baselines are being produced on separate hardware. Our row uses the 200-epoch teacher and is above every published ResNet-18 result on this dataset on *both* axes; Table 18 collects those and Appendix H the details.

Method	Teacher	Clean	AA	Avg	NRR
PGD-AT	–	–	–	–	–
TRADES	–	–	–	–	–
MART	–	–	–	–	–
Consistency-AT	–	–	–	–	–
PGD-AT + WA	–	–	–	–	–
PGD-AT + WA + AWP	–	–	–	–	–
PGD-AT, init. at the teacher	nat.	–	–	–	–
ARD	nat.	–	–	–	–
RSLAD	nat.	–	–	–	–
AdaAD	nat.	–	–	–	–
AdaAD + IGDM	nat.	–	–	–	–
Logit anchor + our stack	nat.	–	–	–	–
LBGAT	nat.	–	–	–	–
ARREST	nat.	–	–	–	–
HAT	nat.	–	–	–	–
ADR	self	–	–	–	–
CURE	self	–	–	–	–
RPAT++	–	–	–	–	–
CFA (ours)	nat.	**55.16**	**20.54**	37.85	**29.93**

Table 4: WideResNet-34-10, ℓ_∞, $\epsilon = 8/255$. Same rows and protocol as Table 2; this architecture is being run on separate hardware, our CIFAR-100 row included. Comparison lines are in Table 17.

		CIFAR-10			CIFAR-100		
Method	T.	Clean	AA	NRR	Clean	AA	NRR
PGD-AT	–	–	–	–	–	–	–
TRADES	–	–	–	–	–	–	–
MART	–	–	–	–	–	–	–
Consistency-AT	–	–	–	–	–	–	–
PGD-AT + WA	–	–	–	–	–	–	–
PGD-AT + WA + AWP	–	–	–	–	–	–	–
PGD-AT, init. at the teacher	nat.	–	–	–	–	–	–
ARD	nat.	–	–	–	–	–	–
RSLAD	nat.	–	–	–	–	–	–
AdaAD	nat.	–	–	–	–	–	–
AdaAD + IGDM	nat.	–	–	–	–	–	–
Logit anchor + our stack	nat.	–	–	–	–	–	–
LBGAT	nat.	–	–	–	–	–	–
ARREST	nat.	–	–	–	–	–	–
HAT	nat.	–	–	–	–	–	–
ADR	self	–	–	–	–	–	–
CURE	self	–	–	–	–	–	–
RPAT++	–	–	–	–	–	–	–
CFA (ours)	nat.	**88.67**	**55.29**	**68.11**	*in progress*		

Tiny-ImageNet, and the teacher dial on a third dataset. Our row uses the 200-epoch teacher. The 80-epoch one gives $57.08/18.96$ — more clean accuracy, less robustness — which is the teacher-length trade-off of Table 12 reproducing on a dataset it was not measured on. Against the literature, the highest published clean accuracy on this dataset for a ResNet-18 is HAT's 52.60 and the highest AutoAttack is 20.23 (AT + WA + ADR), against our $55.16/20.54$.

WideResNet comparison lines. On CIFAR-10 the strongest published pair on either criterion is AT + WA + AWP at $87.42/55.01$ (Avg 71.22, NRR 67.53); on CIFAR-100 it is ADR's $62.21/31.60$ (NRR 41.91) against ARREST's $73.05/24.32$ (Avg 48.69). Both are in Table 17.

WideResNet, CIFAR-10. At $88.67/55.29$ the shipped recipe is strictly better on *both* axes than 30 of the 31 published WideResNet-34-10 results we collected (Table 17), and it takes both criteria: Avg 71.98 against the previous best 71.22 and NRR 68.11 against 67.53, both held by AT + WA + AWP, while its AutoAttack accuracy of 55.29 exceeds the highest published figure of 55.26. The single row it does not dominate is ARREST, which holds 1.57 more standard accuracy and 5.09 less AutoAttack accuracy and therefore occupies a different point on the frontier rather than a better one. The most relevant comparison is with the two methods that also take a naturally trained network: we gain 0.45 standard accuracy and 2.43 AutoAttack accuracy over LBGAT, and 5.09 AutoAttack accuracy over ARREST at 1.57 less standard accuracy. Both of those methods purchase standard accuracy with robustness, whereas the anchor does not.

Reproductions, and one failure. We reproduce RPAT++ to within 0.31 points of AutoAttack accuracy of its published numbers on both datasets and report our own measurement. We were unable to reproduce CURE, as seven runs of the official repository fall short on different axes, and Appendix K gives the detail. Its row is therefore left empty rather than filled from the paper. Published figures for every method here, including the ones we did not run, are collected in Table 17.

5.3 Published Distillation Objectives, Given a Natural Teacher

Table 5: The four published distillation objectives given the teacher they would have in our setting: our naturally trained network in place of the adversarially trained one they assume. Every number is measured here, each method at its own published recipe. Blank cells are runs in progress.

	CIFAR-100			CIFAR-10		
	Clean	AA	NRR	Clean	AA	NRR
ARD	57.61	20.24	29.96	84.58	43.75	57.67
RSLAD	59.68	21.30	31.40	85.15	42.91	57.06
AdaAD	59.79	23.19	33.42	85.39	46.22	59.98
AdaAD + IGDM	48.25	19.48	27.75	85.01	45.50	59.27
PGD-AT, initialized at the teacher	57.73	**26.46**	36.29	82.23	**50.72**	62.74
CFA (ours), no WA/AWP	61.29	25.36	35.88	–	–	–

All four objectives land below the row that does not distil at all, since plain adversarial training initialized from the same teacher beats every one of them on both datasets. The accuracy a natural teacher holds is therefore not recovered by treating that teacher as a logit target.

The IGDM row is reported at $\alpha = 1$ because its published $\alpha = 20$ does not train here at all, collapsing to chance accuracy on both datasets (1.00 on CIFAR-100 and 10.00 on CIFAR-10). The collapse is not a porting artifact. With a natural teacher the IGDM target $\mathrm{softmax}(f_t(x{+}\delta) - f_t(x{-}\delta))$ places 0.917 of its mass on a single class and the term reaches $17.3\times$ the AdaAD loss. At $\alpha = 1$, which puts the two terms on comparable scales and is the most favourable reading available, the method trains normally and still finishes below AdaAD alone on both datasets, by 11.54 clean and 3.71 AutoAttack on CIFAR-100 and by 0.38 and 0.72 on CIFAR-10. Table 8 gives the structural reason: IGDM's finite difference is a gradient only while the teacher is locally linear, and a naturally trained teacher is forty times further from linear than the ones it was designed for.

Table 6: Component ablation. CIFAR-100, ResNet-18, $\epsilon_{\text{train}} = 8.8/255$ fixed. Each row adds one component to the row above, and every rung inherits the teacher's classifier without training it, so the 100-epoch ladder ends on the row Table 2 reports. **The 50-epoch block is the earlier build, in which the head is trained**, and is being rebuilt; read deltas within a block. Per-attack values are in Table 16.

	Clean	AA	Avg	NRR
50 epochs				
Anchor ($p = 0$)	61.33	26.19	43.76	36.71
+ sensitivity-matched ϵ	62.94	26.40	44.67	37.20
+ weight averaging	61.11	**28.06**	44.59	**38.46**
+ AWP†	59.90	28.05	43.98	38.21
100 epochs				
Anchor ($p = 0$)	61.29	25.36	43.33	35.88
+ sensitivity-matched ϵ	**63.09**	25.74	44.42	36.56
+ weight averaging	62.30	27.69	45.00	38.34
+ AWP†	62.17	**28.86**	**45.52**	**39.42**

5.4 Component Ablation

The sensitivity rule contributes $+0.49$ NRR at 50 epochs and $+0.69$ at 100 ($+1.61/+0.21$ and $+1.80/+0.38$ on the two axes) at identical total attack budget. AWP appears to change sign with schedule length — -0.25 NRR at 50 epochs against $+1.08$ at 100 — but the two blocks are not yet the same design, so that reading is held until the 50-epoch rebuild lands. Even so, 50 epochs with weight averaging alone already exceeds ADR's full stack (38.46 against 38.08), at half the schedule and one fewer component, so the gain is not the stack.

Leaving the teacher's classifier alone beats every alternative, including the best temperature. At the full recipe the same holds—freezing the head scores $62.65/28.77/\text{NRR}\ 39.43$ against $62.35/28.68/39.29$ with the head-KD term retained—and this is what removes τ and β from the method. Both arms of that comparison predate the AWP correction described in Appendix L and are quoted as measured, so the pair is internally consistent; the corrected number for the frozen-head arm is the $62.17/28.86/39.42$ reported everywhere else.

Table 7: Two controls on the anchor itself. CIFAR-100, ResNet-18, base regime—teacher warm start, no weight averaging, no AWP, $p = 0$, $\epsilon_{\text{train}} = 8.8/255$—so that no stack component can absorb the difference. Each row differs from its reference in exactly one line of configuration.

	Clean	PGD-20	CW	AA	NRR
(a) where the teacher is read (50 epochs)					
Anchor, Φ_t read at x	61.33	32.94	27.80	**26.19**	**36.71**
read at x_{adv} instead	**76.11**	0.00	0.00	**0.00**	0.00
(b) the anchor against label CE, same regime (100 epochs)					
Anchor	**61.29**	**28.23**	**27.19**	**25.36**	**35.88**
label cross-entropy instead	54.60	20.60	21.21	19.45	28.68
label cross-entropy, own schedule	56.66	22.46	22.91	20.91	30.55

Where the anchor's robustness comes from. Table 7(a) moves Φ_t from x to x_{adv} in both the loss and the attack and changes nothing else. Robustness is then not degraded but eliminated, at 0.00 under PGD-20, CW and AutoAttack alike, while clean accuracy rises by 14.78 points to within 1.55 of the natural teacher. The value of the objective explains the outcome, as it falls to 6.26 against roughly 50.7 for the anchor: once Φ_t is evaluated inside the ball, $\Phi_s = \Phi_t$ minimizes Equation (7) exactly, and that solution is a natural model. Reading the teacher at the clean point is therefore what separates this objective from a degenerate one.

What the anchor is worth on its own. The comparison against cross-entropy in Table 2 carries weight averaging and AWP on both rows, so it does not report what the anchor does without them.

Table 7(b) supplies that comparison at the base regime of the anchor, with the same teacher initialization, the same schedule, the same training radius, no sensitivity-matched ϵ, no weight averaging, no AWP and the classifier inherited and untrained as the shipped recipe leaves it. Replacing the anchor with label cross-entropy under those conditions costs 6.69 points of standard accuracy and 5.91 of AutoAttack accuracy. Our schedule is not the best one for cross-entropy, however, so the row below it gives cross-entropy its own optimizer and schedule instead (SGD at 0.1, decayed by ten at epochs 70 and 90, which is what PGD-AT uses here) while keeping the teacher initialization and the training radius. Cross-entropy gains 2.06 standard accuracy and 1.46 AutoAttack accuracy from the change, and the anchor's lead against that stronger row is 4.63 and 4.45, which is the comparison we read everywhere in this paper. The anchor is therefore a source of robustness rather than merely compatible with it, which is more than Proposition 1 on its own licenses, and it is the larger source at matched stack. For scale, the shipped recipe leads cross-entropy with weight averaging and AWP by 2.40 points of AutoAttack accuracy.

Feature or logit: does the stack close the gap? Table 9 showed the feature anchor beating the best logit temperature in the base regime, by 3.33 points of standard accuracy and 1.40 of AutoAttack accuracy. Weight averaging is what KD + SWA pairs distillation with, so the gap could plausibly close once the stack is added, in which case the contribution would be the schedule rather than the target. We therefore swap only the loss in the shipped recipe, using logit KL at the temperature that maximizes AutoAttack accuracy and leaving everything else untouched, which gives $56.97/26.96$ against $62.17/28.86$. The gap widens with the stack rather than closing, from $3.33/1.40$ to $5.20/1.90$, so the components amplify what the target does rather than substitute for it. The temperature also ceases to be a free parameter under the stack, as $\tau = 16$ trails $\tau = 4$ by 0.48 points of AutoAttack accuracy without it and collapses to $42.45/20.10$ with it. PGD-20 is the one metric that prefers the logit target (33.68 against 32.37), for the reason Appendix I gives: it ascends the cross-entropy through a head the logit objective has trained and the feature objective has left alone.

Is the gain the recipe rather than the objective? Our cells warm-start from the natural teacher and train with AdamW at 0.021 under a one-cycle schedule, and the baselines of Table 2 run at their own published recipes, so the result could be read as a recipe that suits us rather than an objective that works. Three measurements answer that reading and are collected in Table 14. The warm start is worth little, as the anchor trained from random initialization reaches $60.81/25.21$ against $61.29/25.36$ warm-started. The schedule is worth little to the anchor and something to cross-entropy: moving both to PGD-AT's own optimizer and schedule moves the anchor by -0.11 NRR and cross-entropy by $+1.87$, and the anchor leads on either schedule. Handing our recipe to a baseline does not help it, since AdaAD given our optimizer, our schedule, our warm start and our training radius falls to $56.51/22.43$ from the $59.79/23.19$ it reaches at its own recipe.

Is the robustness real? Our objective reads neither labels nor logits, which makes obscured gradients a natural suspicion, and the reported AutoAttack number is decided by two white-box attacks. We therefore run the checks of Athalye et al. (2018) on the shipped cell (Appendix G): the black-box Square attack is 4.80 points weaker than the strongest white-box attack rather than stronger, accuracy falls monotonically in both the radius and the number of steps and reaches exactly zero at $64/255$, and adversarial examples built on the natural teacher cost the student 1.58 points even though it inherits that teacher's classifier.

Where that robustness comes from, and where it does not. The natural explanation is local stability, since Proposition 1 bounds the oscillation O of the student by the anchor's own loss, so that an anchored student should move less inside the ball than one trained on labels. The prediction is not borne out. Attacking both models of Table 7(b) with the same true-label PGD and measuring the feature displacement they actually incur, the anchored student moves 0.275 of its own feature norm against 0.254 for the cross-entropy student, at cosines 0.9626 and 0.9671. The two are indistinguishable on this measurement and the ordering, if anything, favours cross-entropy, so Proposition 1 holds without being the mechanism.

What separates the two models is where the features sit rather than how far they travel, measured on the same models under the same attack:

	clean			adversarial		
	S_w/S_b	to own μ	to nearest μ	S_w/S_b	to own μ	to nearest μ
Anchor	**0.898**	**28.1**°	**28.6**°	**2.681**	**33.3**°	16.2°
Label CE	4.960	45.4°	21.6°	7.143	47.0°	18.3°

The classes of the anchored student are $5.5\times$ better separated on clean data and $2.7\times$ better separated under attack, and the improvement holds on both halves of the ratio: its clusters are tighter ($28.1°$ against $45.4°$ to the class mean) and its class means are further apart ($28.6°$ against $21.6°$). The cross-entropy student sits twice as close to the nearest wrong class mean as to its own, at $21.6°$ against $45.4°$. The geometry of the anchored student degrades under attack, from 0.898 to 2.681, and even then remains better than the undisturbed geometry of the cross-entropy student.

Class separation is the same quantity that Section 5.5 finds varying along the teacher ladder, measured here within a single pair of runs instead of across teachers. A perturbation has to carry a feature across a decision boundary and separation determines how far that is, and the anchored student inherits the separation of a teacher that was never asked to be flat and therefore never spent capacity on flatness.

One alternative reading requires its own measurement, since cross-entropy without weight averaging robustly overfits under our schedule and part of the gap could be decay rather than deficit. Cross-entropy does overfit, as its PGD-20 accuracy peaks at 23.31 near epoch 70 and decays to 21.13, whereas the anchor rises monotonically to the final evaluation ($25.23 \to 28.86 \to 30.72 \to 31.47$). Measured against the best epoch of cross-entropy rather than its last, the final anchor still leads by 8.16 points. Not overfitting is itself a property of the objective here, and it is the property that weight averaging is normally bought to supply.

5.5 Training Radius, Teacher Length, and Larger Architectures

The training radius is a plateau, not a tuned point. Sweeping $\epsilon_{\text{train}} \in \{8, 8.8, 10\}/255$ with everything else fixed and evaluating at $8/255$ throughout (Table 10), NRR peaks at $8.8/255$ on both datasets and the curve around the peak is mild, and the standard $8/255$ already exceeds the best published result on either dataset (38.72 against ADR's 38.08, and 64.48 against CURE's 63.19). Beyond $8.8/255$ the radius buys nothing, since AutoAttack and CW accuracy at $10/255$ match those at $8.8/255$ to the last decimal on both datasets while standard accuracy falls by 2.66 and 2.29 points, which is the ϵ-independence that Appendix B predicts above the threshold.

What the teacher transfers is geometry, not accuracy. Changing only the teacher checkpoint and leaving the student configuration untouched (Table 12), teacher accuracy and student accuracy are anti-correlated at $r = -0.999$ across five teachers trained for 50 to 300 epochs, so that the teacher gains 2.51 points while the student loses 2.04. NRR peaks in the middle of the ladder, which makes the training length of the teacher a control on the operating point, and locating that peak costs five natural-training runs and no adversarial ones. The effect reproduces on Tiny-ImageNet where teacher accuracy is controlled to 0.32 points, so accuracy cannot be the cause; what varies monotonically is the teacher's class separation.

Larger architectures. WideResNet-34-10 at the same recipe on both CIFAR datasets is reported in Table 4, alongside the published WideResNet rows.

6 Conclusion

Adversarial training discards standard accuracy that a naturally trained copy of the same network still holds. We asked whether self-distillation can put it back without touching where the robustness comes from, and found that it can, provided the teacher is asked only for accuracy and is read only at the clean point. Anchoring the student's adversarial feature to the teacher's clean feature makes that precise: the teacher never enters the perturbation ball, so its own instability—$63.8°$ of rotation under $\epsilon = 8/255$—has no path into the objective, and the single term bounds fidelity and local stability together to within a factor of three. The result is a method with no loss weight, no temperature and no trained classifier, whose robustness is produced by the student's own inner maximization rather

than borrowed from the teacher, and which transfers across three datasets with only the teacher checkpoint changed.

Two findings sit alongside it. The first is that consuming the same natural teacher the way adversarial distillation does, as a logit target or read inside the ball, is worse than not distilling at all on both datasets and for all four published objectives. The second is that the teacher transfers its class geometry rather than its accuracy, with the two anti-correlated at $r = -0.999$ across training lengths, which turns the teacher's own schedule into a trade-off control that costs nothing to exercise.

Robustness is still produced by the student's inner maximization and by nothing the teacher supplies, since the teacher is never evaluated under attack. What we did not expect is that *which* objective that maximization ascends matters as much as it does. Stripped of weight averaging, of AWP and of the sensitivity-matched radius, the anchor alone leads label cross-entropy under an identical schedule and initialization by 4.45 points of AutoAttack as well as 4.63 of standard accuracy, read against cross-entropy at its own optimizer and schedule, and 1.80 of that robustness margin is traceable to attacking in feature space rather than through the label. For scale, at the full stack the anchor leads cross-entropy *with* weight averaging and AWP by 2.40. The anchor is therefore not a neutral passenger on machinery that supplies robustness; at matched stack it is the larger contributor, and the stack is additive on top. Proposition 1 licenses only the weaker half of this—that the anchor cannot cost robustness, since bounding one scalar bounds fidelity and local stability together instead of trading them. What the proposition does not give is the size of the effect, and a bound on that is the obvious next step; the measurements that would constrain it are reported here rather than set aside.

A Algorithm

Algorithm 1 Clean Feature Anchoring (CFA)

Input: dataset $\mathcal{D}$, radius ϵ, attack steps K, step size a, exponent p, box $[w_{\mathrm{lo}}, w_{\mathrm{hi}}]$
Output: robust student $h_t \circ \Phi_s$

{*Phase 1 — natural self-teacher.* The only stage that uses labels.}
Train $f_t = h_t \circ \Phi_t$ on $\mathcal{D}$ with standard cross-entropy; freeze it

{*Phase 2 — anchored adversarial training.* No labels, no head update.}
$\Phi_s \leftarrow \Phi_t$ {student initialized at the teacher}
for epoch $= 1$ **to** T **do**
  for minibatch $\{x_i\}_{i=1}^{N} \subset \mathcal{D}$ **do**
    $\phi_i \leftarrow \Phi_t(x_i)$ {teacher read once, at the clean input}
    $g_i \leftarrow \|\nabla_x \|\Phi_s(x_i) - \phi_i\|^2\|$ {one backward pass}
    $r_i \leftarrow (\bar{g}/g_i)^p$; solve $\sum_i \mathrm{clip}(tr_i, w_{\mathrm{lo}}, w_{\mathrm{hi}}) = N$ for t {Equation (18)}
    $\epsilon_i \leftarrow \epsilon \cdot \mathrm{clip}(tr_i, w_{\mathrm{lo}}, w_{\mathrm{hi}})$ $\{\sum_i \epsilon_i = N\epsilon\}$
    $x_i^0 \leftarrow x_i + \eta_i$
    for $k = 0$ **to** $K - 1$ **do**
      $x_i^{k+1} \leftarrow \Pi_{B(x_i, \epsilon_i)}\big(x_i^k + a \operatorname{sign} \nabla_x \|\Phi_s(x_i^k) - \phi_i\|^2\big)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{N} \sum_i \|\Phi_s(x_i^K) - \phi_i\|^2$
    $\Phi_s \leftarrow \Phi_s - \mu \nabla_{\Phi_s} \mathcal{L}$ {h_t is never updated}
  end for
end for

B Two Partial Results on the Robustness Side

Guiding adversarial training with a naturally trained network is not new and we do not claim it: LBGAT, ARREST and DP-FAT all do it. What is different here is that they keep the adversarial label loss and add the natural teacher on top of it, so their robustness is bought by the label term and the anchor is free to contribute clean accuracy alone. Nobody needs to ask whether their teacher's fragility leaks into the student, because the term that makes the student robust is not the teacher.

We delete the label loss. Equation (7) is the whole backbone objective, so the robustness has to come from the inner maximization alone, against a target supplied by a network with 0 AutoAttack accuracy. That makes an otherwise idle objection load-bearing: a representation assembled out of features an adversary can flip is being handed to a student as the only thing it is trained on, and the student might reasonably be expected to inherit the fragility along with the answer. The first result below says that in the model where the objection is usually posed, it does not. The second is a certificate that tracks robustness without predicting it. Neither closes the question the paper actually sells — how much robustness the objective *buys* — and both are reported with the reason they fall short.

B.1 In the Two-Feature Model, the Non-Robust Coefficient Is Zeroed Rather Than Shrunk

The setting is the two-feature model of Tsipras et al. (2018), which is where the objection is usually posed. One feature x_1 agrees with the label with probability $p = 0.95$ and cannot be attacked. A further d features $z_j \sim \mathcal{N}(\eta y, 1)$ are individually almost uninformative, are jointly close to perfectly informative through their mean $m(z)$, and can all be moved by an ℓ_∞ adversary. A naturally trained network therefore reads $m(z)$: near-perfect clean accuracy, no robustness whatever. That is our teacher's profile, so $\Phi_t = m(z)$.

Write the student's feature as $\Phi_s = ax_1 + b\,m(z)$. Then b is exactly the quantity the objection is about: how much of the teacher's fragile route the student takes over. If the objection is right, minimizing our objective should leave b large.

Proposition 2. *With the inner maximum exact, $J(a,b) = \mathbb{E}[R^2] + 2\epsilon|b|\,\mathbb{E}|R| + \epsilon^2 b^2$ with $R = ax_1 + (b-1)m$, kinked at $b = 0$ with subgradient jump $2\epsilon\,\mathbb{E}|R(a,0)| > 0$; above a threshold ϵ_0 the minimizer has $b^\star = 0$ exactly.*

Proof. The adversary reaches Φ_s only through m, and only through the mean perturbation $\bar{\delta} = \frac{1}{d}\sum_j \delta_j$, which ranges over $[-\epsilon, \epsilon]$ as $\|\delta\|_\infty \le \epsilon$. The inner objective is therefore $\max_{|\bar{\delta}|\le\epsilon}(R + b\bar{\delta})^2 = (|R| + \epsilon|b|)^2$, since a scalar quadratic is maximized at an endpoint and the sign of $\bar{\delta}$ is free. Taking expectations and expanding the square gives the display.
Fix a and differentiate in b. The term $\mathbb{E}[R^2]$ and $\epsilon^2 b^2$ are smooth, while $2\epsilon|b|\,\mathbb{E}|R|$ contributes $\pm 2\epsilon\,\mathbb{E}|R|$ as $b \to 0^\pm$. The one-sided derivatives at $b = 0$ are thus $2\mathbb{E}[R(a,0)\,m] \pm 2\epsilon\,\mathbb{E}|R(a,0)|$, so $b = 0$ is a minimum along b exactly when

$$\big|\mathbb{E}[R(a,0)\,m]\big| \;\le\; \epsilon\,\mathbb{E}|R(a,0)|. \tag{13}$$

At $b = 0$ the objective in a is $\mathbb{E}(ax_1 - m)^2$, minimized at $a^\star = \mathbb{E}[x_1 m]/\mathbb{E}[x_1^2] = q\eta$, and there $\mathbb{E}[R(a^\star,0)\,m] = q^2\eta^2 - (\eta^2 + \frac{1}{d}) = -c$ with

$$c \;=\; \eta^2 v + \tfrac{1}{d}, \qquad v = 1 - q^2. \tag{14}$$

Equation (13) therefore holds for every

$$\epsilon \;\ge\; \epsilon_0 \;=\; \frac{c}{\mathbb{E}|q\eta x_1 - m|}, \tag{15}$$

which is the stated threshold. The mechanism is visible in Equation (13): the left side is fixed while the right grows linearly in ϵ, so the kink's slope overtakes the quadratic gain at a finite radius rather than asymptotically. □

Why the coefficient is deleted rather than shrunk. The adversary contributes $2\epsilon|b|\,\mathbb{E}|R|$ to the loss — linear in $|b|$, not quadratic in b. A quadratic penalty pushes a coefficient toward zero and never arrives; a penalty on $|b|$ is kinked at the origin and sets the coefficient to zero outright once its slope dominates, which is the same distinction that makes the lasso sparse and the ridge not. What produces the kink here is that the student is asked to reproduce a *value* rather than a route: the fragile features carry that value on clean data but not inside the ball, so past a certain radius carrying any of them costs more than it pays. The teacher supplies what to match, and the inner maximization discards how the teacher got there.

At $d = 512$ the model's constants give $c = 7.89 \times 10^{-3}$ and $\mathbb{E}|q\eta x_1 - m| = 0.0529$, hence $\epsilon_0 = 0.843\,\eta$ — inside the bracket $(0.5\eta,\ 1.0\eta)$ that the Monte-Carlo minimization returns, and far below the 2η at which the model is usually stated.

Two caveats are worth stating rather than leaving to be found. First, Equation (15) is a condition for $b = 0$ to minimize along b at $a = a^\star$; J is not jointly convex, because $|b|\,\mathbb{E}|R|$ is a product of two convex factors, so global minimality is what the numerical check below supplies and not what the argument proves. Second, the model assumes x_1 is unperturbable and z fully perturbable, so *some* suppression of z is built in before anything is shown. What is not built in is exactness, the threshold, and the ϵ-independence that follows it.

That the exactness is the kink's doing, and not a general consequence of adding an adversary, is visible from dropping it. With the $|b|$ term removed, $J_{\text{quad}} = \mathbb{E}[R^2] + \epsilon^2 b^2$; minimizing out a and writing $u = 1 - b$ collapses it to $u^2 c + \epsilon^2 (1-u)^2$, so

$$b^\star_{\text{quad}} \;=\; \frac{c}{c + \epsilon^2}, \tag{16}$$

which is strictly positive at every radius. Against the exact minimizer ($d = 512$, Monte-Carlo, $n = 2 \times 10^5$):

ϵ	$b^\star$ (exact)	$b^\star_{\text{quad}}$ (kink dropped)
0.5η	0.510	0.502
1.0η	**0.000**	0.202
2.0η	**0.000**	0.060
4.0η	**0.000**	0.016

The two columns agree below ϵ_0 and separate above it: the quadratic surrogate shrinks the non-robust coefficient and never deletes it, which is the ordinary picture, while the kinked objective sets it to zero and keeps it there.

Since every ϵ in J multiplies b, at $b^\star = 0$ the objective is $\mathbb{E}(ax_1 - m)^2$ and is ϵ-free—past the threshold a larger radius costs the anchor nothing, which matches the saturation measured in Table 10, where AutoAttack and CW at $10/255$ are identical to the last decimal to those at $8.8/255$ on both datasets.

This model rules out the paper's effect by construction, so Proposition 2 cannot be the explanation of it: its reliability spectrum has two atoms and no mass near $\eta_j/\epsilon = 1$, where the effect lives. It is a statement about the free half only.

B.2 What Robust Accuracy Tracks Is the Loss Measured Against a Margin

Writing $\gamma_t(x)$ for the teacher's margin at x and D_y for the relevant decision-boundary constant, Cauchy–Schwarz gives that $\gamma_t(x) > D_y \cdot L(x)$ implies the student is robustly correct at x. The certificate is vacuous at the scales we train at, but the ratio DL/γ_t tracks both axes across the teacher ladder of Table 12. What it does not do is predict the size of the effect, and it saturates before AutoAttack does.

C Additional Results

The tables below are referred to from the main text and are placed here for space. They are the same measurements under the same protocol; nothing is reported here that is not stated in the main body.

Table 8: Local linearity, which IGDM's construction requires of the teacher: remainder proportion of the first-order Taylor expansion over an $\epsilon = 8/255$ ball, computed as in IGDM's Sec. 3.1.

Network	Remainder proportion
IGDM's robust teachers (LTD / BDM-AT / IKL-AT, published)	0.012 / 0.012 / 0.016
PGD-AT ResNet-18 (ours)	0.0111
Natural teacher (ours)	**0.4763**
Anchored student, $p = 0$	0.0029
Anchored student, shipped recipe	0.0202

Table 9: What to do with the classifier. CIFAR-100 base regime (50 epochs, raw features, no weight averaging, no AWP, $\epsilon = 8/255$), so that no stack component can absorb the difference.

	Clean	PGD-20	CW	AA	NRR
Logit target only, $\tau = 1$	58.26	22.62	22.79	20.84	30.70
Logit target only, $\tau = 4$	59.39	30.30	26.84	24.48	34.67
Logit target only, $\tau = 16$	57.78	31.57	26.11	24.00	33.91
Feature anchor + head KD, $\tau = 1$	62.34	26.34	26.11	23.93	34.58
Feature anchor + head KD, $\tau = 16$	62.61	32.47	27.32	25.61	36.35
Feature anchor, head untouched	**62.72**	28.69	**27.80**	**25.88**	**36.64**
head refit, adversarial CE	62.77	29.93	27.66	25.65	36.42
head refit, adv. CE + smoothing 0.1	62.57	30.43	27.55	25.66	36.39
head refit, clean CE	62.70	28.90	27.23	25.15	35.90

Table 10: ϵ_{train} swept with everything else fixed. Evaluation is at $\epsilon = 8/255$ in every cell.

	Clean	PGD-20	CW	AA	NRR
CIFAR-100					
$6/255$	**68.07**	28.40	28.16	25.82	37.44
$7/255$	65.61	30.34	29.64	27.44	38.70
$8/255$	63.89	31.41	29.83	27.78	38.72
$8.8/255^{\dagger}$	62.17	32.37	**30.93**	**28.86**	**39.42**
$10/255^{\dagger}$	59.99	**32.82**	30.53	28.77	38.89
CIFAR-10					
$6/255$	**89.64**	49.29	50.20	47.88	62.42
$7/255$	88.30	51.43	52.64	50.27	64.07
$8/255^{\dagger}$	87.22	52.43	53.55	51.15	**64.48**
$8.8/255^{\dagger}$	84.96	52.98	**53.94**	51.74	64.31
$10/255^{\dagger}$	83.29	**53.25**	53.88	**51.79**	63.87

Table 11: What the per-sample radius is allocated *from*. CIFAR-100, ResNet-18, the shipped recipe, with the cells differing in one configuration line each and spending the same total budget $N\epsilon_{\text{train}}$. $\text{sd}(w)$ is the spread of the multiplier the attack actually receives, after the box and the mean restoration. Only the sensitivity signal beats allocating nothing at all.

Allocated from	$\text{sd}(w)$	Clean	PGD-20	CW	AA	NRR
Nothing (uniform, $p = 0$)	0	60.42	**33.11**	30.07	28.42	38.66
Loss sensitivity (ours, $p = 1$)	0.32	**62.17**	32.37	**30.93**	**28.86**	**39.42**
Difficulty magnitude (IAAT, CAT)	0.39	61.27	32.45	30.14	28.12	38.55
Difficulty, same weights permuted	0.32	60.90	32.32	29.94	28.06	38.42
Logit margin (MMA)	0.42	61.07	32.46	29.75	27.79	38.20

Table 12: Only the teacher checkpoint changes; the student configuration is untouched. CIFAR-100, ResNet-18. S_w/S_b is the ratio of within-class to between-class scatter on the unit sphere.

Teacher	Teacher clean	Teacher S_w/S_b	Student clean	Student AA	NRR
50 epochs	75.81	1.100	**64.28**	24.38	35.35
100 epochs	76.62	0.982	63.65	25.20	36.11
150 epochs	77.52	0.887	62.93	25.78	36.51
200 epochs	77.65	0.808	62.72	**25.88**	**36.64**
300 epochs	78.32	0.712	62.24	25.80	36.48

D The Allocation Rule and Its Budget Solve

D.1 Where the rule comes from

The objective is optimized over a **pixel** ball, while what it measures is a displacement in feature space, and a fixed pixel radius moves that displacement by very different amounts across samples. The allocation follows from one elementary fact: for a differentiable loss the first-order movement available inside an ℓ_∞ ball of radius e is

$$\max_{\|\delta\|_\infty \le e} \langle \nabla_x \mathcal{L},\, \delta \rangle \;=\; e\, \|\nabla_x \mathcal{L}\|_1, \tag{17}$$

the dual norm of ℓ_∞ being ℓ_1. Equalizing that movement across a batch subject to a fixed total budget $\sum_i \epsilon_i = N\epsilon$ gives $\epsilon_i g_i = \text{const}$, that is $\epsilon_i \propto 1/g_i$ with $g_i = \|\nabla_x \mathcal{L}(x_i)\|_1$. The same allocation is the max–min one: if two samples move by different amounts, shifting budget from the sample that moves more to the sample that moves less raises the minimum, so no unequal assignment is optimal. Taking $\epsilon_i \propto (\bar{g}/g_i)^p$ makes $p = 1$ that allocation, $p = 0$ the uniform-pixel one, and $p \in (0, 1)$ an interpolation.

Two approximations are worth declaring rather than leaving to be found. First, g_i is measured once, by a single backward pass at the *clean* x_i before the attack starts, so it is the sensitivity at the starting point and not along the PGD trajectory. Second, everything above is first order in ϵ. Neither is repaired anywhere in the method; the rule is a budget heuristic with a derivation, not an exactly solved subproblem.

We also do not claim the rule is specific to a directional objective. $\|\nabla_x \mathcal{L}\|_1$ is defined for any loss and the implementation reads whichever loss is configured; on the shipped raw-ℓ_2 anchor it equalizes a feature displacement rather than an angle, which is why the honest name is *sensitivity-matched* ϵ rather than an angular budget. What separates it from IAAT, MMA and CAT is not directionality but the signal: they allocate from a property of the sample, this allocates from the geometry the training loss is written in.

The separation is not one of dispersion, which is worth stating because a rule that simply spread the budget more aggressively would be the simpler explanation of any gain. Measured over training on the shipped configuration, the coefficient of variation of our sensitivity signal has median 0.56, against 0.75 for the difficulty score IAAT and CAT allocate from and 1.33 for the logit margin of MMA. The box of this section compresses all three before they reach the attack, leaving per-sample multipliers of spread 0.32, 0.39 and 0.42 respectively, and the ordering survives the compression. Ours is the least dispersed of the three signals either way, so the comparison in Table 11 is a comparison of which geometry the budget follows.

One discrepancy is ours to declare rather than the reader's to find. Equation (17) gives the ℓ_1 norm and the implementation reads ℓ_2, for no better reason than that the cell which fixed the recipe used it and every logged run since had to stay reproducible against that choice. The two are not the same allocation. Measured over training on the shipped configuration, the ℓ_1 signal's coefficient of variation has median 0.68 against the ℓ_2 signal's 0.56, and the per-sample multiplier the attack receives has spread 0.34 against 0.32 — so the norms really do distribute the budget differently.

They nonetheless land close together. Training the shipped recipe with `featdir_angeps_gnorm` set to 1 and nothing else changed gives $62.16/31.97/30.56/28.57$ (NRR 39.15) against $62.17/32.37/30.93/28.86$ (NRR 39.42): clean accuracy is identical to a hundredth and AutoAttack differs by 0.29, in favour of the norm we ship.

We do not call that a tie. It is twice the seed-to-seed spread of Appendix E, and it is the same order of magnitude as what the allocation buys in the first place: against the uniform radius, $p = 1$ gains $+1.75$ clean and $+0.44$ AutoAttack at ℓ_2 and $+1.74$ and $+0.15$ at ℓ_1. So the rule's benefit survives the norm in direction and almost entirely on the clean axis, while most of its AutoAttack half does not. The honest statement is that the norm is a real choice which we made before we could compare, that the comparison favours what we ship, and that a reader preferring the derived ℓ_1 would still get the clean gain and a third of the robust one.

The permuted row of Table 11 deserves its construction spelled out, since it is what makes the comparison a comparison of signals rather than of budgets. It takes the multiset of weights the gradient rule produces — the same values, the same clip $[0.5, 1.5]$, the same restored mean — and only permutes which example receives which, so that the larger radii go to the easier examples, which is the direction IAAT and CAT assign them. Reordering costs 0.80 AutoAttack against the rule it reorders and lands below even the uniform row on AutoAttack, CW and NRR alike.

D.2 Keeping the radii in the box

With $r_i = (\bar{g}/g_i)^p$ and box $[w_{\mathrm{lo}}, w_{\mathrm{hi}}] = [0.5, 1.5]$, the radii that satisfy both the box and the budget exactly are $\epsilon_i = \epsilon \cdot \mathrm{clip}(t r_i, w_{\mathrm{lo}}, w_{\mathrm{hi}})$ with t the solution of

$$\sum_{i=1}^{N} \mathrm{clip}\big(t\, r_i,\ w_{\mathrm{lo}},\ w_{\mathrm{hi}}\big) \ =\ N. \tag{18}$$

The left side is continuous and nondecreasing in t from $N w_{\mathrm{lo}}$ to $N w_{\mathrm{hi}}$, so a root exists and bisection finds it; when no clip is active it reduces to $t = N/\sum_i r_i$, that is, to restoring the mean. Clipping first and rescaling afterwards, the obvious alternative, does not preserve the box—the rescaling moves the clipped entries back out of it.

E Experimental Details

Every number we report is the **final** epoch, never a checkpoint selected on a robustness metric. This matters when reading Table 17: RPAT selects on PGD-20 and ADR on PGD-10, so those rows are upper envelopes of a training run where ours is its endpoint.

Teacher. One naturally trained network per dataset, no adversarial examples anywhere in its training, and the student is warm-started from the same checkpoint. ResNet-18, SGD at learning rate 0.1 under a one-cycle schedule, batch size 128, no augmentation beyond the standard crop and flip: 200 epochs on CIFAR-10 and CIFAR-100, 80 on Tiny-ImageNet-200. The resulting teachers reach 95.33 (CIFAR-10), 77.66 (CIFAR-100) and the Tiny-ImageNet figure quoted in Table 3. The teacher's own robustness is 0 — it is never attacked and is not meant to survive attack; Table 12 is the ablation over how long it is trained.

Student. The three shipped cells differ only where the dataset forces them to.

	CIFAR-10	CIFAR-100	Tiny-ImageNet-200
Backbone	ResNet-18	ResNet-18	ResNet-18
Optimizer	AdamW	AdamW	AdamW
Learning rate / schedule	0.021, one-cycle	0.021, one-cycle	0.021, one-cycle
Epochs	100	100	100
Batch size	128	128	128
Training radius $\epsilon_{\mathrm{train}}$	$8.8/255$	$8.8/255$	$8.8/255$
Inner attack	PGD-10, step $2/255$	PGD-10, step $2/255$	PGD-10, step $2/255$
Allocation exponent p	1	1	1
Radius box $[w_{\mathrm{lo}}, w_{\mathrm{hi}}]$	$[0.5, 1.5]$	$[0.5, 1.5]$	$[0.5, 1.5]$
Classifier head	frozen	frozen	frozen
Weight averaging (start, decay)	0.2, 0.999	0.2, 0.999	0.2, 0.999
AWP (γ, warmup)	0.005, 10 ep	0.005, 10 ep	0.005, 10 ep
Teacher checkpoint	clean, 200 ep	clean, 200 ep	clean, 80 ep

The head being frozen is what removes τ and β from the method: with the classifier inherited and never updated, the head-KD term of earlier drafts has no parameters to reach and is switched off entirely. p is not tuned. $\epsilon_{\text{train}} = 8.8/255$ is the one value swept, in Table 10, and evaluation is at $8/255$ in every cell regardless.

Evaluation. AutoAttack, `standard` version (APGD-CE and APGD-T), ℓ_∞ at $\epsilon = 8/255$, run on the full test set. PGD-20 and CW_∞ are reported alongside because they disagree with each other in a way that matters (Appendix I), and NRR is the harmonic mean of clean accuracy and AutoAttack accuracy. Where a table reports a method we ported rather than quoted, the port runs on its own published recipe — HAT and LBGAT both collapse under our schedule and are given theirs — and only the evaluation is ours.

Seeds, and how large a difference has to be. Every number in this paper is one seed. The ablations are read as differences of a few tenths, so one repeat is reported to say what a tenth is worth: the shipped CIFAR-100 cell at seed 1 gives $62.00/32.49/30.82/28.74$ against seed 0's $62.17/32.37/30.93/28.86$, moving each metric by 0.11 to 0.17 and AutoAttack by 0.12. Differences at that scale are not read as differences anywhere in this paper; the component deltas of Table 6 and the gaps of Table 11 are between four and fifteen times it. One repeat is one repeat, and it bounds nothing — it is reported because a reader otherwise has no scale at all.

Compute, teacher included. A single RTX 5080. One student cell is about 2.2 hours on CIFAR-100 at 100 epochs, of which the AutoAttack evaluation is roughly 7 minutes; Tiny-ImageNet is longer in proportion to its test set.

The teacher has to be counted, since calling it free would otherwise hide a training run. It is not free, but it is cheap: a clean epoch has no inner maximization, so it costs 6.8 seconds against 74.5 for an adversarial one, and 200 clean epochs come to 0.39 hours. The teacher is therefore 19% of one student run and is amortized over every cell that reads it, the teacher ladder of Table 12 included. Table 13 puts the total against the baselines measured on the same machine.

Table 13: Wall-clock training cost on one RTX 5080, CIFAR-100, ResNet-18, as logged by the runs reported in this paper. Evaluation is excluded. Adversarial distillation additionally requires an adversarially trained teacher, which is not counted in its row and is the more expensive artifact.

Method	Epochs	Teacher (h)	Student (h)	Total (h)
Natural teacher (ours, reused)	200 clean	–	0.39	0.39
CFA (ours)	100 adv.	0.39	2.07	**2.46**
Label CE at our regime	100 adv.	0.39	3.22	3.61
PGD-AT, initialized at the teacher	100 adv.	0.39	1.72	2.11
ARD	100 adv.	0.39	1.52	1.91
RSLAD	100 adv.	0.39	1.71	2.10
AdaAD	100 adv.	0.39	2.79	3.18
LBGAT	100 adv.	joint	1.88	1.88
HAT	50 adv.	0.39	1.10	1.49
ADR	200 adv.	self	3.52	3.52

Two readings follow. The anchor is not bought with compute, since it costs less end to end than label cross-entropy at the same regime and about what PGD-AT costs once the teacher is included. And the comparison with adversarial distillation is favourable in a way the table understates, because those rows assume a robust teacher that the current literature obtains from a WideResNet trained on generated data, which is more expensive than every row here combined.

F The Recipe Controls

Every anchored cell in this paper warm-starts from the natural teacher and trains with AdamW at 0.021 under a one-cycle schedule, while the baselines of Table 2 run at their authors' recipes. Two objections follow from that asymmetry, and Table 14 answers both. The first is that our recipe is what wins; the second is that the baselines would win with it. Every row is measured here at the

base regime, which carries no weight averaging, no AWP and a uniform radius, on CIFAR-100 with ResNet-18 for 100 epochs.

Table 14: The recipe controls. *Ours* is AdamW at 0.021 under a one-cycle schedule with the teacher warm start; *theirs* is SGD at 0.1 decayed by ten at epochs 70 and 90, which is PGD-AT's schedule here and the one the distillation baselines use. The training radius and every other field are held fixed within each pair.

Objective	Recipe	Clean	AA	Avg	NRR
Anchor	ours, teacher init	**61.29**	25.36	**43.33**	**35.88**
Anchor	ours, random init	60.81	25.21	43.01	35.66
Anchor	theirs, teacher init	59.65	**25.54**	42.60	35.77
Label cross-entropy	ours, teacher init	54.60	19.45	37.03	28.68
Label cross-entropy	theirs, teacher init	56.66	20.91	38.79	30.55
AdaAD	ours, teacher init	56.51	22.43	39.47	32.11
AdaAD	theirs, random init	59.79	23.19	41.49	33.42
RSLAD	ours, teacher init	54.93	19.44	37.19	28.72
RSLAD	theirs, random init	59.68	21.30	40.49	31.40

The anchor moves by 0.22 NRR between initializations and by 0.11 between schedules, against a seed-to-seed spread of 0.12 measured in Appendix E, so neither is where its result comes from. Cross-entropy moves by 1.87 NRR between the two schedules, which is the more important number: our schedule is not the best one for cross-entropy, and the base-regime comparison in Table 7 is therefore read against the stronger of its two rows. AdaAD loses 1.31 NRR when given our recipe and RSLAD loses 2.68, so the recipe is not a general improvement that the baselines were denied.

G Gradient Masking

The AutoAttack numbers reported in this paper run APGD-CE followed by APGD-T on the full test set. Both are white-box gradient attacks, and the objective we train reads neither labels nor logits, so the question of whether the robustness is an artifact of obscured gradients has to be answered rather than asserted. Table 15 runs the checks of Athalye et al. (2018) on the shipped CIFAR-100 cell. FAB-T and Square are evaluated on the first 1000 test examples, which is the usual convention for the two expensive attacks; everything else uses all 10000.

Table 15: Gradient-masking checks on the shipped CIFAR-100 cell (clean accuracy 62.00). Each attack is run independently at $\epsilon = 8/255$, so the numbers are standalone rather than the cascade AutoAttack reports. The teacher-transfer row builds PGD-20 examples on the natural teacher and evaluates them on the student, which shares the teacher's classifier.

Check	Setting	Accuracy
white box against black box, $n = 1000$		
APGD-CE	white box, untargeted	30.20
APGD-T	white box, targeted	**28.00**
FAB-T	white box, boundary	28.70
Square	**black box**, 5000 queries	32.80
radius sweep, PGD-20, $n = 10000$		
$\epsilon = 2/255$		54.62
$\epsilon = 4/255$		47.03
$\epsilon = 8/255$		32.46
$\epsilon = 12/255$		20.88
$\epsilon = 16/255$		11.94
$\epsilon = 32/255$		1.05
$\epsilon = 64/255$		**0.00**
step sweep, $\epsilon = 8/255$, $n = 10000$		
PGD-10 / 20 / 50 / 100		33.17 / 32.57 / 32.30 / 32.24
transfer, $n = 10000$		
Natural teacher $\rightarrow$ student	PGD-20 built on the teacher	60.42

Four things follow. The black-box attack is the weakest of the four, by 4.80 points against the strongest white-box one, where masked gradients produce the opposite ordering. Accuracy falls monotonically with the radius and reaches exactly zero at $64/255$, so there is no radius at which the attack stops working. Accuracy falls monotonically with the number of steps and flattens by 50, rather than oscillating. And APGD-T, which is the attack our reported number is decided by, is the strongest of the four here.

The transfer row is worth reading separately. The student inherits the teacher's classifier and is initialized at the teacher's features, so adversarial examples built on the teacher might be expected to carry over; they cost the student 1.58 points against its clean accuracy. Sharing a head is therefore not sharing a vulnerability, which is the same conclusion Section 5.4 reaches from class separation: what the anchored student changes is where features sit, and the teacher's own adversarial directions do not point at the student's boundaries.

H Notes on the Baselines

Several rows of Table 2 run a specific variant of the method they name, or are grouped in a way that needs saying. This section is those decisions; the table carries none of them.

LBGAT, and why its CIFAR-10 cell is empty. The authors release a training script for CIFAR-100 only, and their CIFAR-10 entries are released as pretrained WideResNet checkpoints. Running their CIFAR-100 recipe on CIFAR-10 diverges here within thirty steps: the squared error is reduced with `mean` over batch and classes, so a ten-class problem receives ten times the gradient of a hundred-class one at equal logit error, and $\mathrm{lr} = 0.1$ is above the stability threshold that follows. Lowering the learning rate trains (at 0.01 the natural branch reaches 49% within 180 steps), but a learning rate we chose is not their recipe, so the cell is left empty rather than filled with a number of our own construction. LBGAT trains its natural branch jointly with the robust one, so unlike the distillation rows its teacher is not obtained for free, and we group it accordingly. The two datasets run different variants because the authors publish different ones: CIFAR-100 is LBGAT6 and CIFAR-10 is LBGAT0, their release lists CIFAR-10 checkpoints under LBGAT0 alone, and $\beta = 6$ is their CIFAR-100 setting. We report the configuration each dataset has a published counterpart for rather than forcing one β across both.

HAT uses a naturally trained network only to *label* helper points beyond the ball, never as a target, so it belongs with the trade-off methods and not the distillation ones. It is run on its own one-cycle schedule rather than ours, which collapses it to chance on both datasets — the same allowance we make for LBGAT. On CIFAR-10 the port reproduces HAT's published no-extra-data figure to within 0.31 clean and 0.27 AutoAttack ($84.90/49.08$ reported, ResNet-18), which is what licenses reporting its CIFAR-100 row at all: HAT publishes no CIFAR-100 result without additional data, its $61.50/28.88$ being trained with DDPM-generated samples, so that column has no published figure to be read against.

ADR is run on separate hardware and its row is filled from there. Two things have to hold when it is. ADR reports two configurations — AT + ADR at $56.10/26.87$ on CIFAR-100 and AT + WA + AWP + ADR at $57.36/28.50$ — and the row has to name which, the second being the one component-matched to our recipe, which carries both weight averaging and AWP. ADR is also self-distillation: its teacher is an exponential moving average of the student, no external network is read, and the student trains from scratch, so its row inherits nothing of our natural teacher.

What the published tables are blocked by. Table 17 and Table 18 separate their rows by network, and the separations are not cosmetic. RPAT's "ResNet-18" is `ResNet(Bottleneck, [2,2,2,2])` in its released code, 14.35M parameters against the 11.27M of the standard `BasicBlock` network used by us and by ADR; they also ship a `pre_resnet18` at 11.27M and use PreActResNet-18 explicitly elsewhere, so the distinction is theirs. HAT's Tiny-ImageNet row is PreActResNet-18: ADR lists it under ResNet-18, but the figures are quoted from HAT's own Table 9, whose caption says PreAct, and HAT's code refuses anything else on that dataset — same parameter count as ours, different network, and its two companions from that table are shown beside it. F^{2}AT's plain SAT baseline reaches $31.52/9.37$ where ADR's plain AT reaches $45.87/18.06$, a

gap no difference of method explains, so its rows are separated too. TE's architecture we could not verify — it is ADR's attribution to a paper we do not have — and it is marked accordingly. ADR trains Tiny-ImageNet for 80 epochs against our 100, which favours us and is stated here rather than left implicit. LBGAT reports that dataset without AutoAttack and B-MTARD reports it on MobileNet-v2, so neither appears; CURE, ARREST and Generalist++ do not report it at all.

IGDM is reported at $\alpha = 1$. Its published $\alpha = 20$ diverges to chance accuracy on both datasets when the teacher is natural, for the reason given in Section 5.3.

Logit anchor + our stack is not a published method. It is our own recipe with the feature anchor replaced by logit KL — the closest thing to KD + SWA (Chen et al., 2021) that a natural teacher admits, distil then average weights, with a logit target where ours is a feature one — and it exists to isolate the target from the schedule. Its temperature is chosen in the baseline's favour rather than faithfully: plain distillation is $\tau = 1$, which is the worst of the three in Table 9 at 20.84 AutoAttack, and the row uses the τ that maximizes it, 4 at 24.48. The comparison is therefore against the best logit target we could find rather than the one the name implies. Temperature also stops being a small effect once the stack is present: the two sit 0.48 apart without it, and $\tau = 16$ reaches only $42.45/20.10$ with it, against $\tau = 4$'s $56.97/26.96$.

I Per-Attack Metrics

Table 16 gives every attack we measure for our own cells. The main tables report clean accuracy and AutoAttack because those are the two axes the trade-off is defined on, and because PGD-20 is not a faithful reading of this method.

Table 16: Per-attack accuracy (%) for our cells. ResNet-18, $\epsilon = 8/255$, final-epoch weight-averaged model, full test set.

Dataset	Cell	Clean	FGSM	PGD-20	CW	AA	NRR
CIFAR-100	CFA, head frozen (shipped)	62.17	36.91	32.37	30.93	28.86	39.42
	the same cell at seed 1	62.00	36.79	32.49	30.82	28.74	39.27
	ℓ_1 gradient norm	62.16	36.48	31.97	30.56	28.57	39.15
	CFA, head-KD term retained	62.35	38.80	36.26	30.65	28.68	39.29
	CFA, $\epsilon_{\text{tr}} = 6/255$	68.07	35.88	28.40	28.16	25.82	37.44
	CFA, $\epsilon_{\text{tr}} = 7/255$	65.61	36.32	30.34	29.64	27.44	38.70
	CFA, $\epsilon_{\text{tr}} = 8/255$	63.89	36.67	31.41	29.83	27.78	38.72
	CFA, $\epsilon_{\text{tr}} = 10/255$	59.99	36.50	32.82	30.53	28.77	38.89
	CFA, no WA / AWP	61.29	33.97	28.23	27.19	25.36	35.88
	Logit anchor, our stack	56.97	35.82	33.68	28.94	26.96	36.60
	RPAT++ (our reproduction)	55.93	34.46	32.44	29.37	27.36	36.74
CIFAR-10	CFA, $\epsilon_{\text{tr}} = 8.8/255$ (shipped)	84.96	59.58	52.98	53.94	51.74	64.31
	CFA, $\epsilon_{\text{tr}} = 6/255$	89.64	59.37	49.29	50.20	47.88	62.42
	CFA, $\epsilon_{\text{tr}} = 7/255$	88.30	59.59	51.43	52.64	50.27	64.07
	CFA, $\epsilon_{\text{tr}} = 8/255$	87.22	59.92	52.43	53.55	51.15	64.48
	CFA, $\epsilon_{\text{tr}} = 10/255$	83.29	58.98	53.25	53.88	51.79	63.87
	RPAT++ (our reproduction)	82.41	59.73	55.72	52.88	50.75	62.82

Why PGD-20 is not the metric here. The student inherits the teacher's classifier and never retrains it. PGD-20 ascends the cross-entropy *through that classifier*, so it reads a head that was fitted for a different feature distribution; AutoAttack, which includes a targeted APGD, and CW_∞, which attacks the logit margin directly, do not depend on that fit in the same way. The consequence is measurable on a controlled pair. The first two rows of Table 16 share a backbone objective, a schedule and an attack, and differ only in whether the classifier is refit:

	PGD-20	CW	AA
head-KD term retained	36.26	30.65	28.68
head frozen	32.63	30.66	28.77
difference	-3.63	$+0.01$	$+0.09$

Changing the classifier moves PGD-20 by 3.63 points and moves CW and AutoAttack by hundredths. The same pattern recurs across our ablations—the head-temperature sweep, the label-smoothing sweep, the directional-versus-raw comparison, and the radius sweep all show PGD-20 moving against CW and AutoAttack. We therefore report PGD-20 for completeness and read the trade-off on clean accuracy and AutoAttack, which is also what ADR and ARREST report in their main tables.

J Published Results, for Context

Table 2 contain only numbers we measured ourselves. Table 17 collects the published figures for the same and neighbouring methods, so that our runs can be placed against the literature without the two being mixed in one column. These are quoted as reported and were produced by other codebases, other schedules and other checkpoint-selection rules—RPAT selects its best checkpoint by PGD-20 and ADR by PGD-10, where every number of ours is the final epoch—so they should be compared within a source rather than across.

Read against Table 2 and restricted to the ResNet-18 blocks, which are the ones our numbers are comparable with, our CFA row exceeds every published Avg and NRR on both datasets: $62.17/28.86$ at Avg 45.52 on CIFAR-100 and $84.96/51.74$ at Avg 68.35 on CIFAR-10.

The restriction is not a convenience. The WideResNet-34-10 block sits above us on both criteria—RPAT++ reaches Avg 70.87 and NRR 67.30 there—because a 48.3M network is a different model, not a better method, and Table 4 is where that comparison belongs. The same caution applies to the PreActResNet-18 blocks in the other direction: RPAT++ falls to $82.63/51.00$ there, and reading that as a defeat for RPAT would be the identical error. We state all of this as a comparison across codebases and architectures rather than as a controlled result, which is what Table 2 is for.

Table 17: Published results, ℓ_∞, $\epsilon = 8/255$, quoted as reported. **The architecture changes between blocks** and is named in each heading; only the ResNet-18 blocks are comparable with Table 2. These were produced by other codebases, schedules and checkpoint-selection rules, so they should be compared within a source rather than across.

Method	Teacher	Clean	AA	Avg	NRR	Source
CIFAR-100, ResNet-18						
PGD-AT	–	56.56	25.02	40.79	34.69	RPAT
TRADES	–	55.39	24.51	39.95	33.98	RPAT
MART	–	49.83	25.00	37.41	33.30	RPAT
Consistency-AT	–	58.53	25.39	41.96	35.42	RPAT
Consistency-AT + RPAT	–	60.33	26.31	43.32	36.64	RPAT
F^2AT	–	54.19	23.24	38.71	32.53	F^2AT
Generalist++	co-tr.	62.97	23.96	43.47	34.71	Generalist++
ADR (AT + ADR)	self	56.10	26.87	41.48	36.34	ADR
ADR (AT + WA + AWP + ADR)	self	57.36	28.50	42.93	38.08	ADR
CIFAR-10, ResNet-18						
PGD-AT	–	82.78	44.63	63.71	57.99	CURE
TRADES	–	82.41	48.37	65.39	60.96	CURE
MART	–	80.70	47.49	64.10	59.79	CURE
ST-AT	–	83.10	50.50	66.80	62.82	CURE
IAD	robust	80.63	50.17	65.40	61.85	CURE
Consistency-AT + RPAT	–	84.12	48.98	66.55	61.91	RPAT
Generalist++	co-tr.	89.09	46.07	67.58	60.73	Generalist++
ARREST	natural	86.63	46.14	66.39	60.21	ARREST
CURE	self	86.76	49.69	68.23	63.19	CURE
ADR (AT + ADR)	self	82.41	50.38	66.40	62.53	ADR
CIFAR-10, PreActResNet-18						
WA (Izmailov et al., 2018)	–	83.50	49.89	66.70	62.46	RPAT
AWP (Wu et al., 2020)	–	81.11	50.09	65.60	61.93	RPAT
KD + SWA (Chen et al., 2021)	robust	84.06	49.82	66.94	62.56	RPAT
TE (Dong et al., 2022)	–	82.04	50.12	66.08	62.23	RPAT
ReBAT (Wang et al., 2023a)	–	82.09	50.72	66.41	62.70	RPAT
RPAT++ (Liu et al., 2025)	–	82.63	51.00	66.82	63.07	RPAT
CIFAR-100, PreActResNet-18						
WA (Izmailov et al., 2018)	–	57.26	25.83	41.55	35.60	RPAT
AWP (Wu et al., 2020)	–	54.10	25.16	39.63	34.35	RPAT
KD + SWA (Chen et al., 2021)	robust	57.17	25.66	41.42	35.42	RPAT
TE (Dong et al., 2022)	–	56.41	25.84	41.13	35.44	RPAT
ReBAT (Wang et al., 2023a)	–	56.13	27.60	41.87	37.00	RPAT
RPAT++ (Liu et al., 2025)	–	56.84	27.68	42.26	37.23	RPAT
CIFAR-10, WideResNet-34-10						
WA (Izmailov et al., 2018)	–	87.66	52.65	70.16	65.79	RPAT
MMA (Ding et al., 2020)	–	87.80	43.10	65.45	57.82	RPAT
AWP (Wu et al., 2020)	–	85.63	53.32	69.48	65.72	RPAT
KD + SWA (Chen et al., 2021)	robust	87.45	53.59	70.52	66.46	RPAT
TE (Dong et al., 2022)	–	85.97	52.88	69.43	65.48	RPAT
ReBAT (Wang et al., 2023a)	–	85.25	54.78	70.02	66.70	RPAT
ADR (Wu et al., 2024)	self	84.67	53.25	68.96	65.38	RPAT
CURE (Gowda et al., 2024)	self	87.05	52.10	69.58	65.19	RPAT
RPAT++ (Liu et al., 2025)	–	86.76	54.97	70.87	67.30	RPAT

Table 18: Published Tiny-ImageNet-200 results, ResNet-18, ℓ_∞, $\epsilon = 8/255$, quoted as reported. Our row is above all of them on both axes. The blocks are separated because the networks are not the same, and Appendix H says how each differs.

Method	Clean	AA	Avg	NRR	Source
CFA (ours)	**55.16**	**20.54**	**37.85**	**29.93**	—
AT + WA + ADR	48.55	20.23	34.39	28.56	ADR
AT + WA + AWP + ADR	48.27	20.12	34.20	28.40	ADR
TRADES + WA + AWP + ADR	51.38	19.48	35.43	28.25	ADR
TRADES + WA + ADR	51.99	19.17	35.58	28.01	ADR
AT + WA + AWP	48.61	19.58	34.09	27.92	ADR
AT + WA	49.10	19.30	34.20	27.71	ADR
TE*	47.46	18.29	32.88	26.40	ADR
AT	45.87	18.06	31.96	25.92	ADR
TRADES	48.49	17.35	32.92	25.56	ADR
PreActResNet-18 — same parameter count, different network					
HAT	52.60	18.14	35.37	26.98	HAT
TRADES	48.25	17.17	32.71	25.35	HAT
AT	47.76	17.92	32.84	26.02	HAT
Bottleneck ResNet-18 (14.35M) and an unstated setup — see caption					
Consistency-AT + RPAT	49.74	18.84	34.29	27.33	RPAT
PGD-AT + RPAT	47.68	17.77	32.73	25.89	RPAT
Consistency-AT	46.54	17.60	32.07	25.54	RPAT
MART	39.70	17.18	28.44	23.98	RPAT
F^2AT	40.54	13.13	26.84	19.84	F^2AT
SEAT	35.51	11.37	23.44	17.22	F^2AT
SAT	31.52	9.37	20.45	14.45	F^2AT

K THE CURE REPRODUCTION

Seven runs of the official CURE repository, from its own code and our shared CIFAR data, none of which reproduce the published $86.76/49.69$ on CIFAR-10. They fall short on *different* axes: the closest on standard accuracy reaches 86.11 at AA 40.65, the closest on robustness AA 45.63 at clean 81.15, while the published pair has both. Running the repository unmodified gives $65.03/29.74$. Computing its RGP prominence mask on absolute values raises AA by 15.9 points ($29.74 \rightarrow 45.63$), which localizes the discrepancy to that step without closing it.

We re-evaluated all five surviving checkpoints with our own pipeline and it agrees with the repository's own evaluation to within 0.05 AutoAttack on every one, so the gap is in training rather than in measurement. For contrast, RPAT++ reproduces to within 0.31 AutoAttack of its published numbers on both datasets under the same conditions. The CURE row of Table 17 is therefore the paper's own number, and CURE is left empty in Table 2.

L A CORRECTION TO THE AWP PROXY, AND WHAT IT MOVED

AWP ascends an auxiliary objective in weight space before each descent step. In our implementation the closure used for that ascent had drifted from the closure used for descent, in two ways. It retained a head-distillation term that `freeze_head` removes from training, so AWP was maximizing a quantity partly composed of a term the model never optimizes; and a detach intended for the normalized feature was applied on a path that feeds the unnormalized one. Measured on CIFAR-100 before the fix, the spurious term was 7.15 against the anchor's 4.15 and contributed 40.8% of the AWP backbone gradient, at cosine 0.918 to the correct direction. Separately, the head was frozen *after* the proxy was copied, which is harmless only because the proxy's first step is delayed by the AWP warmup.

We report this because it is the kind of defect that changes a paper's numbers, and because here it did not. Re-running the shipped recipe with both closures agreeing gives $62.17/28.86$ against $62.65/28.77$ on CIFAR-100 and $84.96/51.74$ against $85.58/51.79$ on CIFAR-10: NRR moves by 0.01 and 0.22, no ordering in any table changes, and clean accuracy moves more than robustness

does. AWP is evidently insensitive to a large, well-aligned perturbation of its ascent direction, which is consistent with its role here as a regularizer of sharpness rather than a precisely targeted objective. All numbers reported for the shipped recipe are post-correction. Two exceptions are stated where they appear: the head-freeze comparison (in Section 5.4 and again in Appendix I), the component ladder in Table 6, and the ϵ_{train} sweep in Table 10 were measured before it. In each, every arm shares the defect, so the comparison is internally consistent even though its absolute values sit a few hundredths from the corrected ones.

REFERENCES

Anonymous. Toward understanding adversarial distillation. In *ICML*, 2026.

Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *International conference on machine learning*, pp. 274–283. PMLR, 2018.

Yogesh Balaji, Tom Goldstein, and Judy Hoffman. Instance adaptive adversarial training. In *arXiv*, 2019.

Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 ieee symposium on security and privacy (sp)*, pp. 39–57. Ieee, 2017.

Erh-Chung Chen and Che-Rung Lee. LTD: low temperature distillation for robust adversarial training. *CoRR*, abs/2111.02331, 2021. URL `https://arxiv.org/abs/2111.02331`.

Tianlong Chen, Zhenyu Zhang, Sijia Liu, Shiyu Chang, and Zhangyang Wang. Robust overfitting may be mitigated by properly learned smoothening. In *International Conference on Learning Representations*, 2021.

Minhao Cheng, Qi Lei, Pin-Yu Chen, Inderjit Dhillon, and Cho-Jui Hsieh. Customized adversarial training for improved robustness. In *IJCAI*, 2022.

Seungju Cho and Changick Kim. Direction-preserving fast adversarial training. In *under review*, 2026.

Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In *International conference on machine learning*, pp. 2206–2216. PMLR, 2020.

Jiequan Cui, Shu Liu, Liwei Wang, and Jiaya Jia. Learnable boundary guided adversarial training. In *ICCV*, 2021.

Gavin Weiguang Ding, Yash Sharma, Kry Yik Chau Lui, and Ruitong Huang. MMA training: Direct input space margin maximization through adversarial training. In *ICLR*, 2020.

Yinpeng Dong, Ke Xu, Xiao Yang, Tianyu Pang, Zhijie Deng, Hang Su, and Jun Zhu. Exploring memorization in adversarial training. In *International Conference on Learning Representations*, 2022.

Micah Goldblum, Liam Fowl, Soheil Feizi, and Tom Goldstein. Adversarially robust distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 3996–4003, 2020.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.

Shruthi Gowda, Bahram Zonooz, and Elahe Arani. Conserve-update-revise to cure generalization and robustness trade-off in adversarial training. In *ICLR*, 2024.

Sorin Grigorescu, Bogdan Trasnea, Tiberiu Cocias, and Gigel Macesanu. A survey of deep learning techniques for autonomous driving. *Journal of Field Robotics*, 37(3):362–386, 2020.

Bo Huang, Mingyang Chen, Yi Wang, Junda Lu, Minhao Cheng, and Wei Wang. Boosting accuracy and robustness of student models via adaptive adversarial distillation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 24668–24677, 2023.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In *Conference on Uncertainty in Artificial Intelligence*, 2018.

Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

Ya Le and Xuan Yang. Tiny imagenet visual recognition challenge. *CS 231N*, 7(7):3, 2015.

Hongsin Lee, Seungju Cho, and Changick Kim. Indirect gradient matching for adversarial robust distillation. In *ICLR*, 2025.

Yanyun Liu et al. Failure cases are better learned but boundary says sorry: Facilitating smooth perception change for accuracy-robustness trade-off in adversarial training. In *ICCV*, 2025.

Xingjun Ma, Yuhao Niu, Lin Gu, Yisen Wang, Yitian Zhao, James Bailey, and Feng Lu. Understanding adversarial attacks on deep learning based medical image analysis systems. *Pattern Recognition*, 110:107332, 2021.

Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.

Tianyu Pang, Xiao Yang, Yinpeng Dong, Hang Su, and Jun Zhu. Bag of tricks for adversarial training. *arXiv preprint arXiv:2010.00467*, 2020.

Leslie Rice, Eric Wong, and J. Zico Kolter. Overfitting in adversarially robust deep learning. In *ICML*, 2020.

Satoshi Suzuki, Shin'ya Yamaguchi, Shoichiro Takeda, Sekitoshi Kanai, Naoki Makishima, Atsushi Ando, and Ryo Masumura. ARREST: Adversarial finetuning with latent representation constraint to mitigate the accuracy-robustness tradeoff. In *ICCV*, 2023.

Jihoon Tack, Sihyun Yu, Jongheon Jeong, Minseon Kim, Sung Ju Hwang, and Jinwoo Shin. Consistency regularization for adversarial robustness. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 8414–8422, 2022.

Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. Robustness may be at odds with accuracy. *arXiv preprint arXiv:1805.12152*, 2018.

Yisen Wang et al. Robust weight perturbation for adversarial training. In *NeurIPS*, 2023a.

Zekai Wang, Tianyu Pang, Chao Du, Min Lin, Weiwei Liu, and Shuicheng Yan. Better diffusion models further improve adversarial training. In *International Conference on Machine Learning (ICML)*, 2023b.

Dongxian Wu, Shu-Tao Xia, and Yisen Wang. Adversarial weight perturbation helps robust generalization. *Advances in Neural Information Processing Systems*, 33:2958–2969, 2020.

Yu-Yu Wu, Hung-Jui Wang, and Shang-Tse Chen. Annealing self-distillation rectification improves adversarial training. In *ICLR*, 2024.

Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric Xing, Laurent El Ghaoui, and Michael Jordan. Theoretically principled trade-off between robustness and accuracy. In *International conference on machine learning*, pp. 7472–7482. PMLR, 2019.

Shiji Zhao, Xizhe Wang, and Xingxing Wei. Mitigating accuracy-robustness trade-off via balanced multi-teacher adversarial distillation. *IEEE TPAMI*, 2024.

Jianing Zhu, Jiangchao Yao, Bo Han, Jingfeng Zhang, Tongliang Liu, Gang Niu, Jingren Zhou, Jianliang Xu, and Hongxia Yang. Reliable adversarial distillation with unreliable teachers. *arXiv preprint arXiv:2106.04928*, 2021.

Bojia Zi, Shihao Zhao, Xingjun Ma, and Yu-Gang Jiang. Revisiting adversarial robustness distillation: Robust soft labels make student better. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16443–16452, 2021.